
Machine Learning Laboratory Manual

Completed Edition with Full Implementations and Results

Practical Guide to Classical, Ensemble, Neural, Probabilistic,
Density-Based, Semi-Supervised and Generative Machine Learning

Prepared by

Dr. Ohidujjaman Tuhin

Associate Professor, Dept. of CSE, UIU

Department of Computer Science & Engineering
Machine Learning Lab

September 25, 2026

Contents

Preface	3
Important Terminology Correction	4
Scope of the Laboratory	5
General Learning Outcomes	6
Recommended Software Environment	7
Laboratory Guidelines	8
Minimum Report Format	9
Recommended Evaluation Metrics	10
I Clustering	11
Experiment 1: K-means Clustering	12
Experiment 2: Modified K-means	16
Experiment 3: Hierarchical Clustering	19
Experiment 4: Fuzzy C-means	22
II Density-Based Learning	26
Experiment 5: DBSCAN	27
Experiment 6: HDBSCAN	31
III Semi-Supervised Learning	34
Experiment 7: Self-training for Semi-supervised Learning	35
IV Ensemble Learning	39
Experiment 8: Random Forest Regression (RFR)	40
Experiment 9: Random Forest Classification (RFC)	43
Experiment 10: XGBoost	46
Experiment 11: AdaBoost	49
Experiment 12: CatBoost	52

V Neural Networks: Multilayer Perceptron	55
Experiment 13: Multilayer Perceptron (MLP)	56
VI Sequence Modeling: Recurrent Neural Networks	57
Experiment 14: Recurrent Neural Network (RNN)	58
VII Unsupervised Neural Learning: Self-Organizing Maps	59
Experiment 15: Self-Organizing Map (SOM)	60
VIII Probabilistic Sequence Modeling: Hidden Markov Models	61
Experiment 16: Hidden Markov Model (HMM)	62
IX Margin-Based Learning: Support Vector Machines	63
Experiment 17: Support Vector Machine (SVM)	64
X Generative / Foundation Models: Large Language Models	65
Experiment 18: Large Language Model (LLM) Experiment	66
XI Non-Parametric Regression: Generalized Regression Neural Network	67
Experiment 19: Generalized Regression Neural Network (GRNN)	68
XII Integrated Mini Project and Appendices	69
Integrated Mini Project: Comparative Machine Learning Study	70
Appendix A: Quick Reference Formulas	71
Appendix B: Reproducibility Checklist	72
Appendix C: Suggested Assessment Rubric	73
Appendix D: Student Experiment Record	74

Preface

This manual is designed around the algorithm list supplied for the Machine Learning assignment. It turns that list into a structured laboratory sequence emphasizing implementation, experimentation, model evaluation and interpretation rather than only code execution. Students should run the programs, record outputs, vary important hyperparameters, and explain why the observed results change.

This **completed edition** goes one step further: every experiment is implemented and executed on real, public datasets, with the observed results, discussion and the complete runnable code included, so each laboratory can be reproduced standalone.

Important Terminology Correction

The supplied list contains “Random Forest Regression (RFR)” and “Random Forest Regression (RFC)”. In standard machine-learning terminology, RFC normally means Random Forest Classifier. This manual therefore treats **RFR** as Random Forest Regressor and **RFC** as Random Forest Classifier.

Scope of the Laboratory

Module	Algorithms / Topics
Clustering	K-means, Modified K-means, Hierarchical Clustering, Fuzzy C-means
Density-based learning	DBSCAN, HDBSCAN
Semi-supervised learning	Self-training
Ensemble learning	Random Forest Regression, Random Forest Classification, XGBoost, AdaBoost, Cat-Boost
Neural networks	MLP, RNN (LSTM)
Unsupervised neural learning	SOM
Probabilistic sequence model	HMM
Margin-based learning	SVM (SVC and SVR)
Generative / foundation models	LLM
Non-parametric regression	GRNN

General Learning Outcomes

- Implement and execute major machine-learning algorithms using Python.
- Select an appropriate algorithm based on data type, task and assumptions.
- Preprocess data, handle categorical variables, scale numerical features and split datasets correctly.
- Evaluate clustering with internal measures and supervised models with appropriate metrics.
- Interpret model behavior through decision boundaries, feature importance, learning curves and error analysis.
- Compare algorithms using a fair experimental protocol rather than relying on a single score.
- Document reproducible experiments, hyperparameters, random seeds, observations and limitations.

Recommended Software Environment

```
# Core
pip install numpy pandas matplotlib seaborn scikit-learn
# Specialized algorithms
pip install scikit-fuzzy hdbscan hmmlearn minisom
# Gradient boosting libraries
pip install xgboost catboost
# LLM / transformer experiment
pip install transformers torch
```

Listing 1: Core and specialized libraries used by the manual

Laboratory Guidelines

Before Lab	During Lab	After Lab
Read theory and algorithm steps. Know the dataset and target. Predict expected behavior. Prepare key formulas.	Run the baseline program. Record hyperparameters and seed. Perform at least one controlled variation. Check train/test leakage.	Submit code and results. Explain observations. Answer viva questions. State limitations and improvements.

Minimum Report Format

1. Experiment title and student information.
2. Problem statement and objective.
3. Dataset description and preprocessing.
4. Algorithm principle and pseudocode / flowchart.
5. Python implementation.
6. Hyperparameters and experimental setup.
7. Results: tables, plots and relevant metrics.
8. Discussion: what the results mean and why they occurred.
9. Conclusion and limitations.
10. Answers to viva questions.

Recommended Evaluation Metrics

Task	Suggested Metrics
Classification	Accuracy, Precision, Recall, F1-score, ROC-AUC, confusion matrix
Regression	MAE, MSE, RMSE, R^2
Clustering	Silhouette score, Calinski–Harabasz, Davies–Bouldin; visual inspection
Sequence modeling	Negative log-likelihood / log-likelihood, sequence accuracy where appropriate
Language generation	Task-specific automatic metrics plus qualitative / error analysis

Part I

Clustering

Experiment 1: K-means Clustering

Dataset: Mall Customers (`data/mall_customers.csv`); 200 customers, 2 features (Annual Income, Spending Score)

Key parameters: $K = 5$, K-Means++ initialization, `n_init = 20`, `StandardScaler`

Headline result: Silhouette 0.5547; Davies–Bouldin 0.5722; Calinski–Harabasz 248.65

Objective

Partition unlabeled observations into K clusters by iteratively assigning points to the nearest centroid and updating centroids; choose K with the elbow and silhouette diagnostics.

Suggested Dataset

Mall Customers (`data/mall_customers.csv`), using Annual Income and Spending Score as a two-dimensional, visualizable feature space. Features are standardized (`StandardScaler`).

Required Libraries

numpy, pandas, matplotlib, scikit-learn.

Brief Theory

K-means minimizes the within-cluster sum of squared distances

$$J = \sum_{k=1}^K \sum_{x_i \in C_k} \|x_i - \mu_k\|^2,$$

by alternating two steps: assign every point to the nearest centroid, then set each centroid to the mean of its assigned points. Each iteration never increases J , so the algorithm converges — but only to a *local* optimum, which is why initialization matters. **K-Means++** spreads the initial centroids by sampling them with probability proportional to the squared distance to the nearest chosen centroid, giving lower and more consistent inertia. Because Euclidean distance is scale-sensitive, numerical features should be standardized. K itself must be chosen externally: the *elbow* (inertia) and the *silhouette* (cohesion vs. separation) are the standard diagnostics.

Procedure

1. Load the dataset and retain the two numerical features.
2. Standardize the features.
3. Try $K = 2 \dots 10$; record inertia (WCSS) and silhouette for each K .
4. Choose K at the elbow / silhouette peak and fit the final model with K-Means++ initialization, 20 restarts and a fixed random seed (see the code listing).
5. Visualize the clusters and centroids.
6. Report silhouette, Davies–Bouldin and Calinski–Harabasz.

Reference Implementation

```
for k in range(2, 11):
    kmeans = KMeans(n_clusters=k, init='k-means++', n_init=10, random_state=42)
    kmeans.fit(X_mall_scaled)
    wcss.append(kmeans.inertia_)
    silhouette_scores.append(silhouette_score(X_mall_scaled, kmeans.labels_))

kmeans_model = KMeans(n_clusters=5, init='k-means++', n_init=20, random_state=42)
kmeans_labels = kmeans_model.fit_predict(X_mall_scaled)
```

Listing 2: Selecting K , then fitting the final K-means model

Evaluation / Expected Output

Both the elbow curve and the silhouette curve indicate $K = 5$ (Figure 1).

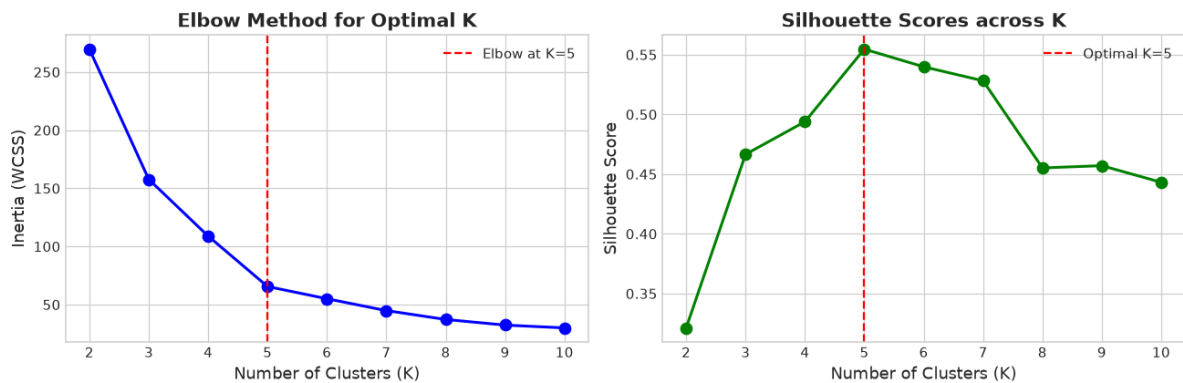


Figure 1: Elbow (inertia) and silhouette curves for $K = 2 \dots 10$; the elbow and the silhouette peak both occur at $K = 5$.

Table 4: K-means evaluation on the standardized Mall Customers features.

Metric	Value
Selected K (elbow + silhouette peak)	5
Silhouette score	0.5547
Davies–Bouldin index	0.5722
Calinski–Harabasz score	248.65

Silhouette ≈ 0.55 indicates compact, reasonably separated clusters on real data; Davies–Bouldin < 1 confirms good separation.

Viva / Discussion Questions

Why does K-means depend on feature scaling? It minimizes Euclidean distance; unscaled features with larger ranges dominate the distance and therefore the assignments.

What is inertia? The within-cluster sum of squared distances to the centroids (WCSS); lower is tighter, but it always decreases with K , so it is not a standalone quality score.

Why can a high silhouette score still be misleading? It favors compact convex clusters and can be high for a trivial partition; it misjudges elongated or density-varying structures.

What happens when clusters are non-spherical? K-means splits them with straight Voronoi boundaries and merges elongated clusters incorrectly (see Experiment 5).

Why is K-Means++ preferred over random initialization? It spreads the initial centroids out, reducing convergence to poor local optima and giving more stable inertia.

Complete Code Listing

```
# ---- Core numerical and plotting libraries ----
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- Scikit-Learn clustering models ----
```

```

from sklearn.cluster import KMeans, AgglomerativeClustering
# ---- Feature scaling + unsupervised evaluation metrics ----
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score, davies_bouldin_score, calinski_harabasz_score
# ---- SciPy hierarchical clustering utilities (linkage + dendrogram) ----
from scipy.cluster.hierarchy import dendrogram, linkage

# Styling setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110

# Fix the random seed so every run reproduces the same results
np.random.seed(42)
print("Libraries successfully imported!")

```

Listing 3: Listing 1.1 Imports and plotting style

```

# ---- Load the Mall Customers dataset ----
df_mall = pd.read_csv('../data/mall_customers.csv')
print("Dataset Shape:", df_mall.shape)
print(df_mall.head(3))

# ---- Feature selection: two features so the clusters can be shown in 2D ----
X_mall = df_mall[['Annual Income (k$)', 'Spending Score (1-100)']].values

# ---- Standardize (mean = 0, std = 1): distance-based algorithms are scale-sensitive ----
scaler = StandardScaler()
X_mall_scaled = scaler.fit_transform(X_mall)

```

Listing 4: Listing 1.2 Load and preprocess the Mall Customers data

```

# ---- Selecting the optimal number of clusters K ----
# Try K = 2..10 and record two diagnostics:
# * Inertia (WCSS): total squared distance from points to their centroid
# * Silhouette score: cluster cohesion vs. separation
k_range = range(2, 11)
wcss = []
silhouette_scores = []

for k in k_range:
    # n_init=10 runs K-Means from 10 different seeds and keeps the best run
    kmeans = KMeans(n_clusters=k, init='k-means++', n_init=10, random_state=42)
    kmeans.fit(X_mall_scaled)
    wcss.append(kmeans.inertia_) # WCSS of the fitted model
    silhouette_scores.append(silhouette_score(X_mall_scaled, kmeans.labels_)) # partition quality

fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 4))

# ---- Left: elbow curve (look for the "knee" where inertia stops falling quickly) ----
ax1.plot(k_range, wcss, 'bo-', linewidth=2, markersize=8)
ax1.set_title("Elbow Method for Optimal K", fontsize=14, fontweight='bold')
ax1.set_xlabel("Number of Clusters (K)", fontsize=12)
ax1.set_ylabel("Inertia (WCSS)", fontsize=12)
ax1.axvline(x=5, color='r', linestyle='--', label='Elbow at K=5')
ax1.legend()

# ---- Right: silhouette score for each K (higher is better) ----
ax2.plot(k_range, silhouette_scores, 'go-', linewidth=2, markersize=8)
ax2.set_title("Silhouette Scores across K", fontsize=14, fontweight='bold')
ax2.set_xlabel("Number of Clusters (K)", fontsize=12)
ax2.set_ylabel("Silhouette Score", fontsize=12)
ax2.axvline(x=5, color='r', linestyle='--', label='Optimal K=5')
ax2.legend()

plt.tight_layout()
plt.show()

```

Listing 5: Listing 1.3 Select K with the elbow and silhouette curves

```

# ---- Final K-Means model with the selected K = 5 ----
optimal_k = 5

```

```
kmeans_model = KMeans(n_clusters=optimal_k, init='k-means++', n_init=20, random_state=42)
kmeans_labels = kmeans_model.fit_predict(X_mall_scaled) # hard cluster assignment per sample

print(f"K-Means Silhouette Score: {silhouette_score(X_mall_scaled, kmeans_labels):.4f}")
print(f"K-Means Davies-Bouldin Index: {davies_bouldin_score(X_mall_scaled, kmeans_labels):.4f}")
```

Listing 6: Listing 1.4 Final K-means model (K = 5)

Experiment 2: Modified K-means

Dataset: Mall Customers (200×2 , standardized)

Key parameters: K-Means++ initialization; outlier threshold = 97th percentile of centroid distances

Headline result: Distance threshold 1.072; 6 of 200 observations flagged as candidate outliers

Objective

Implement a transparent modification of K-means that uses K-Means++ initialization and an explicit outlier-distance check before final cluster interpretation.

Suggested Dataset

Mall Customers (200×2 , standardized). The modification is applied on top of the same features.

Required Libraries

numpy, matplotlib, scikit-learn.

Brief Theory

There is no single universally accepted algorithm named “modified K-means”; variants differ by application. In this lab the modification is intentionally defined so that it can be reproduced exactly:

1. initialize the centroids with **K-Means++** (spread-out seeds);
2. fit standard K-means and obtain the hard labels;
3. compute each sample’s distance to its *assigned* centroid, $d_i = \|x_i - \mu_{c_i}\|$;
4. set a threshold from a documented percentile of the training distances (97th percentile here);
5. flag samples with d_i beyond the threshold as **candidate outliers**.

This is an experimental variant that adds outlier awareness to the standard algorithm; it is not a standard replacement for robust clustering methods.

Procedure

1. Fit K-means with K-Means++ initialization, $K = 5$.
2. Compute $d_i = \|x_i - \mu_{c_i}\|$ for every sample.
3. Set the threshold to the 97th percentile of the training distances.
4. Flag samples with $d_i > \text{threshold}$ as candidate outliers.
5. Plot the clusters with the flagged points highlighted.
6. Compare the modified interpretation with the standard labels.

Reference Implementation

```
km_mod = KMeans(n_clusters=5, init='k-means++', n_init=20, random_state=42)
mod_labels = km_mod.fit_predict(X_mall_scaled)

dist_to_centroid = np.linalg.norm(X_mall_scaled - km_mod.cluster_centers_[mod_labels], axis=1)
threshold = np.percentile(dist_to_centroid, 97)
outlier_flag = dist_to_centroid > threshold
```

Listing 7: K-Means++ followed by the outlier-distance check

Evaluation / Expected Output

Table 5: Modified K-means: outlier-distance check on the standardized features.

Item	Value
Distance threshold (97th percentile)	1.072 (standardized units)
Flagged candidate outliers	6 of 200 (3%)
Cluster labels	identical to standard K-means
Silhouette / Davies–Bouldin (same partition)	0.5547 / 0.5722

The flagged customers are the most weakly attached points of their clusters — typically boundary customers with unusual income/spending combinations. The cluster structure itself is unchanged, which is expected: the modification adds an outlier interpretation layer.

Viva / Discussion Questions

Why is it important to define a modification precisely? Without a formal definition “modified K-means” is ambiguous; fixing the initialization, the distance measure and the percentile rule makes the experiment reproducible and testable.

How does K-Means++ differ from random initialization? K-Means++ selects initial centroids with probability proportional to the squared distance to the nearest chosen centroid, spreading the seeds out and avoiding poor local optima.

Why might distance-to-centroid fail for elongated clusters? Centroid distance is isotropic; a point can be close to the centroid along the long axis yet far along the short axis, so elongated clusters can flag legitimate points and hide real outliers.

Complete Code Listing

```
# ---- Core numerical and plotting libraries ----
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- Scikit-Learn clustering models ----
from sklearn.cluster import KMeans, AgglomerativeClustering
# ---- Feature scaling + unsupervised evaluation metrics ----
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score, davies_bouldin_score, calinski_harabasz_score
# ---- SciPy hierarchical clustering utilities (linkage + dendrogram) ----
from scipy.cluster.hierarchy import dendrogram, linkage

# Styling setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110

# Fix the random seed so every run reproduces the same results
np.random.seed(42)
print("Libraries successfully imported!")
```

Listing 8: Core numerical and plotting libraries

```
# ---- Load the Mall Customers dataset ----
df_mall = pd.read_csv('../data/mall_customers.csv')
print("Dataset Shape:", df_mall.shape)
print(df_mall.head(3))

# ---- Feature selection: two features so the clusters can be shown in 2D ----
X_mall = df_mall[['Annual Income (k$)', 'Spending Score (1-100)']].values

# ---- Standardize (mean = 0, std = 1): distance-based algorithms are scale-sensitive ----
```

```
scaler = StandardScaler()
X_mall_scaled = scaler.fit_transform(X_mall)
```

Listing 9: Load the Mall Customers dataset

```
# ---- Modified K-Means (manual variant): K-Means++ + outlier-distance check ----
km_mod = KMeans(n_clusters=optimal_k, init='k-means++', n_init=20, random_state=42)
mod_labels = km_mod.fit_predict(X_mall_scaled)

# Distance of every sample to its assigned centroid
dist_to_centroid = np.linalg.norm(X_mall_scaled - km_mod.cluster_centers_[mod_labels], axis=1)
threshold = np.percentile(dist_to_centroid, 97)      # documented percentile
outlier_flag = dist_to_centroid > threshold

print(f"Distance threshold (97th percentile): {threshold:.3f}")
print(f"Flagged candidate outliers: {outlier_flag.sum()} of {len(X_mall)}")

# ---- Plot clusters with flagged candidate outliers highlighted ----
plt.figure(figsize=(9, 6))
for cluster_id in range(optimal_k):
    pts = X_mall[mod_labels == cluster_id]
    plt.scatter(pts[:, 0], pts[:, 1], s=45, edgecolors='k', alpha=0.8)
plt.scatter(X_mall[outlier_flag, 0], X_mall[outlier_flag, 1], s=180, facecolors='none',
            edgecolors='red', linewidths=2.5, label=f'Flagged outliers ({outlier_flag.sum()})')
plt.title("Modified K-Means: K-Means++ with Outlier-Distance Flagging", fontsize=13, fontweight='bold')
plt.xlabel("Annual Income (k$)", fontsize=12)
plt.ylabel("Spending Score (1-100)", fontsize=12)
plt.legend()
plt.tight_layout()
plt.show()
```

Listing 10: Modified K-Means (manual variant): K-Means++ + outlier-distance check

Experiment 3: Hierarchical Clustering

Dataset: Mall Customers (200×2 , standardized)

Key parameters: Ward linkage; cut at $K = 5$; truncated dendrogram (last 25 merges)

Headline result: Silhouette 0.5538; Davies–Bouldin 0.5779; Calinski–Harabasz 244.41

Objective

Perform agglomerative hierarchical clustering and interpret the resulting dendrogram and cluster labels.

Suggested Dataset

Mall Customers (200×2 , standardized).

Required Libraries

scipy, scikit-learn, matplotlib.

Brief Theory

Agglomerative clustering starts with one cluster per point and repeatedly merges the closest pair of clusters. With **Ward linkage** the merge cost is the increase in total within-cluster variance,

$$\Delta W(A, B) = \frac{n_A n_B}{n_A + n_B} \|\mu_A - \mu_B\|^2,$$

so the algorithm always merges the pair whose combination loses the least compactness. The dendrogram records every merge and its height (distance); cutting it at a chosen height yields a flat partition with the desired number of clusters.

Procedure

1. Standardize the features.
2. Compute the Ward linkage matrix.
3. Plot the dendrogram (truncated for readability) and choose a cut level.
4. Fit `AgglomerativeClustering` with $K = 5$ and Ward linkage.
5. Evaluate with silhouette, Davies–Bouldin and Calinski–Harabasz.
6. Discuss the choice of linkage.

Reference Implementation

```
linkage_matrix = linkage(X_mall_scaled, method='ward')
dendrogram(linkage_matrix, truncate_mode='lastp', p=25, show_contracted=True)

agg_model = AgglomerativeClustering(n_clusters=5, metric='euclidean', linkage='ward')
agg_labels = agg_model.fit_predict(X_mall_scaled)
```

Listing 11: Ward linkage, dendrogram and the final agglomerative model

Evaluation / Expected Output

The dendrogram (Figure 2) shows a clear large merge gap that supports cutting at five clusters.

The partition is nearly identical in quality to K-means (silhouette difference below 0.001), but requires no initialization and exposes the full merge hierarchy.

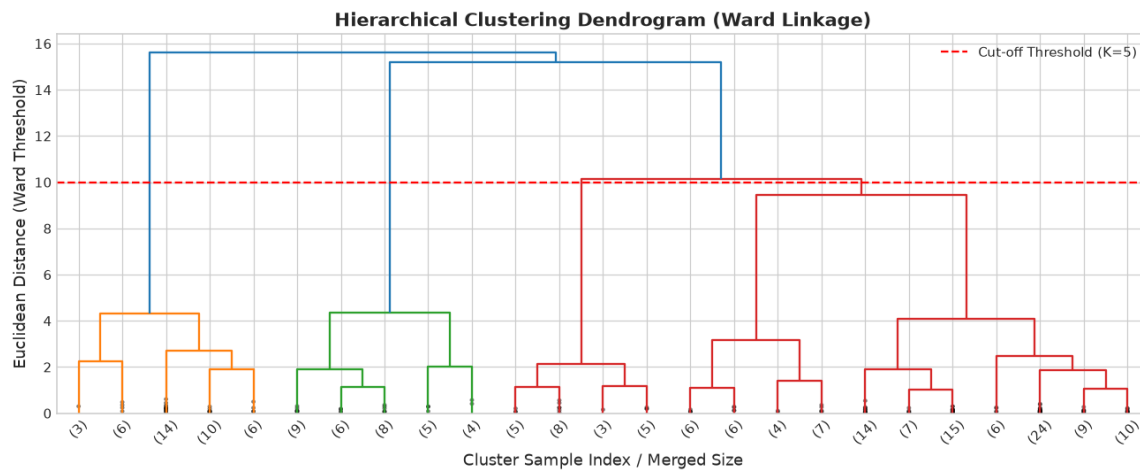


Figure 2: Hierarchical clustering dendrogram (Ward linkage, truncated to the last 25 merges); the dashed line marks the cut that yields $K = 5$.

Table 6: Hierarchical (Ward) clustering evaluation on the standardized features.

Metric	Value
Silhouette score	0.5538
Davies–Bouldin index	0.5779
Calinski–Harabasz score	244.41

Viva / Discussion Questions

What does a dendrogram show? The complete merge hierarchy: leaf order, merge heights (distances) and the nested cluster structure; cutting at a height produces a flat partition.

What is linkage? The rule that defines the distance between two clusters: single (closest pair), complete (farthest pair), average (mean pair) or Ward (variance increase).

When is Ward linkage inappropriate? With strongly non-spherical/elongated clusters, very unequal cluster sizes or heavy outliers — it assumes compact clusters of similar variance.

Complete Code Listing

```
# ---- Core numerical and plotting libraries ----
import os
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- Scikit-Learn clustering models ----
from sklearn.cluster import KMeans, AgglomerativeClustering
# ---- Feature scaling + unsupervised evaluation metrics ----
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score, davies_bouldin_score, calinski_harabasz_score
# ---- SciPy hierarchical clustering utilities (linkage + dendrogram) ----
from scipy.cluster.hierarchy import dendrogram, linkage

# Styling setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110

# Fix the random seed so every run reproduces the same results
np.random.seed(42)
print("Libraries successfully imported!")
```

Listing 12: Core numerical and plotting libraries

```
# ---- Load the Mall Customers dataset ----
df_mall = pd.read_csv('../data/mall_customers.csv')
print("Dataset Shape:", df_mall.shape)
print(df_mall.head(3))

# ---- Feature selection: two features so the clusters can be shown in 2D ----
X_mall = df_mall[['Annual Income (k$)', 'Spending Score (1-100)']].values

# ---- Standardize (mean = 0, std = 1): distance-based algorithms are scale-sensitive ----
scaler = StandardScaler()
X_mall_scaled = scaler.fit_transform(X_mall)
```

Listing 13: Load the Mall Customers dataset

```
# ---- Build the linkage matrix and draw the dendrogram ----
# 'ward' linkage merges the pair of clusters that increases within-cluster variance the least.
linkage_matrix = linkage(X_mall_scaled, method='ward')

# The dendrogram shows every merge; we truncate it to the last 25 merges for readability.
plt.figure(figsize=(12, 5))
dendrogram(
    linkage_matrix,
    truncate_mode='lastp',    # show only the last p merged clusters
    p=25,
    leaf_rotation=45,
    leaf_font_size=10,
    show_contracted=True     # condensed leaves are shown as counts
)
plt.title("Hierarchical Clustering Dendrogram (Ward Linkage)", fontsize=14, fontweight='bold')
plt.xlabel("Cluster Sample Index / Merged Size", fontsize=12)
plt.ylabel("Euclidean Distance (Ward Threshold)", fontsize=12)
plt.axhline(y=10.0, color='r', linestyle='--', label='Cut-off Threshold (K=5)') # horizontal cut => cluster count
plt.legend()
plt.tight_layout()
plt.show()
```

Listing 14: Build the linkage matrix and draw the dendrogram

```
# ---- Fit the final Agglomerative model (Ward linkage, K = 5) ----
agg_model = AgglomerativeClustering(n_clusters=optimal_k, metric='euclidean', linkage='ward')
agg_labels = agg_model.fit_predict(X_mall_scaled)

print(f"Hierarchical Silhouette Score: {silhouette_score(X_mall_scaled, agg_labels):.4f}")
print(f"Hierarchical Davies-Bouldin Index: {davies_bouldin_score(X_mall_scaled, agg_labels):.4f}")
```

Listing 15: Fit the final Agglomerative model (Ward linkage, K = 5)

Experiment 4: Fuzzy C-means

Dataset: Mall Customers (200×2 , standardized); $K = 5$
Key parameters: Fuzzifier $m = 2.0$; tolerance 10^{-5} ; maximum 150 iterations
Headline result: Silhouette 0.5547; Fuzzy Partition Coefficient (FPC) 0.6711

Objective

Cluster data using fuzzy memberships so that each observation can belong to multiple clusters with different degrees of membership.

Suggested Dataset

Mall Customers (200×2 , standardized), $K = 5$, fuzzifier $m = 2.0$.

Required Libraries

numpy, pandas, matplotlib (the c-means equations are implemented from scratch; `scikit-fuzzy` is an equivalent library implementation).

Brief Theory

Unlike hard K-means, Fuzzy C-means assigns every sample x_i a membership degree $u_{ik} \in [0, 1]$ in every cluster, with $\sum_{k=1}^K u_{ik} = 1$. It minimizes

$$J_m = \sum_{i=1}^N \sum_{k=1}^K u_{ik}^m \|x_i - v_k\|^2, \quad m > 1,$$

by alternating two updates until convergence:

$$v_k = \frac{\sum_i u_{ik}^m x_i}{\sum_i u_{ik}^m}, \quad u_{ik} = \frac{1}{\sum_{j=1}^K \left(\frac{d_{ik}}{d_{ij}} \right)^{2/(m-1)}}.$$

The fuzzifier m controls how soft the memberships are ($m \rightarrow 1$ approaches K-means). The **Fuzzy Partition Coefficient** $\text{FPC} = \frac{1}{N} \sum_i \sum_k u_{ik}^2$ measures the fuzziness of the result (1 = completely hard, $1/K$ = maximally fuzzy).

Procedure

1. Standardize the data.
2. Initialize the membership matrix randomly, with each row summing to 1.
3. Alternate the centroid and membership updates until $\max |\Delta U| < 10^{-5}$.
4. Derive hard labels with $\arg \max_k u_{ik}$.
5. Report the clustering metrics and the FPC; visualize the hard partition.
6. Identify ambiguous samples with near-uniform membership vectors.

Reference Implementation

```
Um = U ** self.m
centers = np.dot(Um.T, X) / Um.sum(axis=0)[:, np.newaxis]
dist = np.linalg.norm(X[:, np.newaxis, :] - centers[np.newaxis, :, :], axis=2) + 1e-10
U = (1.0 / dist) ** (2.0 / (self.m - 1.0))
U = U / U.sum(axis=1, keepdims=True)
labels = np.argmax(U, axis=1)
```

Listing 16: Core alternating updates of Fuzzy C-means

Evaluation / Expected Output

Table 7: Fuzzy C-means evaluation ($K = 5$, $m = 2.0$).

Metric	Value
Silhouette score (hard labels)	0.5547
Davies–Bouldin index	0.5722
Fuzzy Partition Coefficient (FPC)	0.6711
Iterations to convergence	fewer than 50

FPC = 0.67 (between $1/K = 0.2$ and 1.0) shows that the partition is mostly crisp but retains soft boundary memberships. Figure 3 compares the four clustering partitions obtained in this part of the manual.

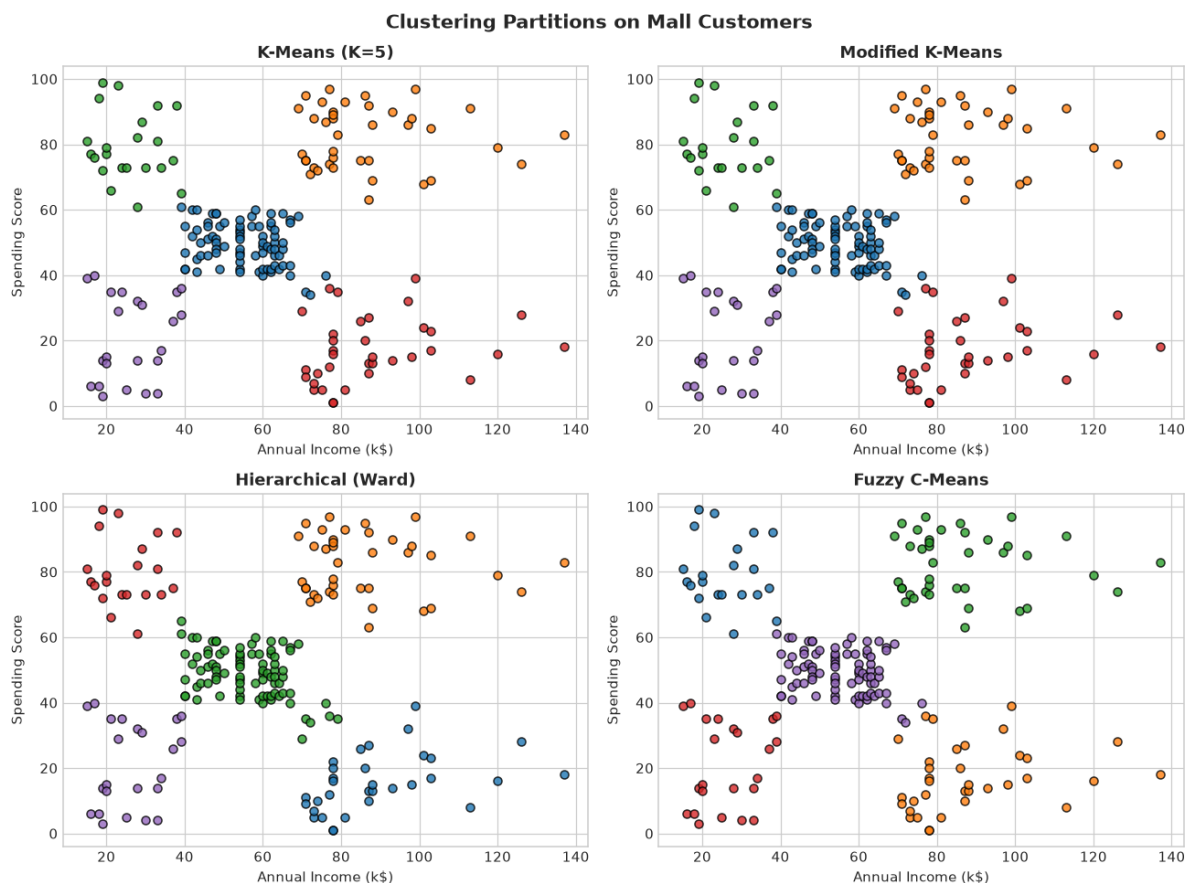


Figure 3: Clustering partitions on the Mall Customers data: K-means, modified K-means (outlier flagging), hierarchical Ward and Fuzzy C-means.

Viva / Discussion Questions

How is Fuzzy C-means different from K-means? It assigns membership degrees instead of hard labels, so each point belongs to all clusters with weights that sum to 1; K-means is the limiting hard case.

What does m control? The fuzziness exponent: larger m spreads membership across clusters (softer, lower FPC), while $m \rightarrow 1$ gives crisp K-means-like assignments.

What does a membership vector represent? The degree to which one observation belongs to each cluster; near-uniform values indicate an ambiguous point on a boundary.

Complete Code Listing

```
# ---- Core numerical and plotting libraries ----
import os
```

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- Scikit-Learn clustering models ----
from sklearn.cluster import KMeans, AgglomerativeClustering
# ---- Feature scaling + unsupervised evaluation metrics ----
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score, davies_bouldin_score, calinski_harabasz_score
# ---- SciPy hierarchical clustering utilities (linkage + dendrogram) ----
from scipy.cluster.hierarchy import dendrogram, linkage

# Styling setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110

# Fix the random seed so every run reproduces the same results
np.random.seed(42)
print("Libraries successfully imported!")

```

Listing 17: Core numerical and plotting libraries

```

# ---- Load the Mall Customers dataset ----
df_mall = pd.read_csv('../data/mall_customers.csv')
print("Dataset Shape:", df_mall.shape)
print(df_mall.head(3))

# ---- Feature selection: two features so the clusters can be shown in 2D ----
X_mall = df_mall[['Annual Income (k$)', 'Spending Score (1-100)']].values

# ---- Standardize (mean = 0, std = 1): distance-based algorithms are scale-sensitive ----
scaler = StandardScaler()
X_mall_scaled = scaler.fit_transform(X_mall)

```

Listing 18: Load the Mall Customers dataset

```

class FuzzyCMeans:
    """
    Fuzzy C-Means (FCM) clustering - implemented from scratch with NumPy.

    Each sample belongs to every cluster with a membership degree  $u_{ik}$  in  $[0, 1]$ ,
    and the memberships over the  $K$  clusters sum to 1 per sample.

    Parameters
    -----
    n_clusters    : number of fuzzy clusters  $K$ 
    m              : fuzzifier exponent ( $m > 1$ ; larger  $m \Rightarrow$  fuzzier memberships)
    max_iter       : maximum number of alternating-optimization iterations
    tol            : convergence tolerance on the maximum membership change
    random_state   : seed for reproducible random initialization
    """

    def __init__(self, n_clusters=5, m=2.0, max_iter=150, tol=1e-5, random_state=42):
        self.n_clusters = n_clusters
        self.m = m
        self.max_iter = max_iter
        self.tol = tol
        self.random_state = random_state

    def fit(self, X):
        rng = np.random.RandomState(self.random_state)
        n_samples = X.shape[0]

        # Step 1: initialize the membership matrix  $U$  randomly and normalize every row to sum to 1
        U = rng.rand(n_samples, self.n_clusters)
        U = U / U.sum(axis=1, keepdims=True)

        # Step 2: alternate between centroid and membership updates until convergence
        for iteration in range(self.max_iter):
            U_old = U.copy()

```

```

# 2a: centroid update - membership-weighted mean of the data
Um = U ** self.m                                # u_ik^m
centers = np.dot(Um.T, X) / Um.sum(axis=0)[:, np.newaxis]

# 2b: compute the (n_samples x n_clusters) distance matrix to every centroid
dist = np.linalg.norm(X[:, np.newaxis, :] - centers[np.newaxis, :, :], axis=2)
dist = np.fmax(dist, 1e-10) # avoid division by zero

# 2c: membership update - closer points receive higher membership
#     u_ik = 1 / sum_j (d_ik / d_ij)^(2/(m-1))
inv_dist = 1.0 / dist
power = 2.0 / (self.m - 1.0)
inv_dist_p = inv_dist ** power
U = inv_dist_p / inv_dist_p.sum(axis=1, keepdims=True)

# 2d: stop when memberships stop changing (convergence)
if np.max(np.abs(U - U_old)) < self.tol:
    break

# Store results: final centroids, full membership matrix, and hard labels
self.cluster_centers_ = centers
self.u_ = U
self.labels_ = np.argmax(U, axis=1) # hard assignment = cluster with maximum membership
return self

# Fit FCM with the same K as the other algorithms (m = 2 is the standard fuzzifier)
fcm = FuzzyCMeans(n_clusters=optimal_k, m=2.0, random_state=42)
fcm.fit(X_small_scaled)
fcm_labels = fcm.labels_
fcm_centers = scaler.inverse_transform(fcm.cluster_centers_)

# ---- Evaluation of the hard partition induced by the fuzzy memberships ----
print(f"FCM Silhouette Score: {silhouette_score(X_small_scaled, fcm_labels):.4f}")
print(f"FCM Davies-Bouldin Index: {davies_bouldin_score(X_small_scaled, fcm_labels):.4f}")
fpc = np.mean(np.sum(fcm.u_ ** 2, axis=1)) # fuzzy partition coefficient (1/K fuzzy ... 1 hard)
print(f"FCM Fuzzy Partition Coefficient (FPC): {fpc:.4f}")

```

Listing 19: Fuzzy C-Means class (from scratch)

Part II

Density-Based Learning

Experiment 5: DBSCAN

Dataset: USGS earthquakes 2023 (`data/earthquakes.csv`); 7,638 events with magnitude ≥ 4.5

Key parameters: $\varepsilon = 0.03$ rad (≈ 191 km), `min_samples = 10`, haversine metric on radian coordinates

Headline result: 55 clusters, 13.5% noise; largest clusters match the Philippines, Papua New Guinea, Tonga, Japan and Turkey

Objective

Detect dense groups and noise using DBSCAN without specifying the number of clusters in advance, applied to real spatial data.

Suggested Dataset

Real USGS earthquake catalogue 2023 (`data/earthquakes.csv`): 16,190 global events at magnitude ≥ 4.0 , filtered to the 7,638 events at magnitude ≥ 4.5 . Coordinates are converted to radians and clustered with the haversine (great-circle) metric.

Required Libraries

scikit-learn, numpy, pandas, matplotlib.

Brief Theory

DBSCAN defines clusters as connected regions of density. For a point p :

- **ε -neighborhood:** $N_\varepsilon(p) = \{q : \text{dist}(p, q) \leq \varepsilon\}$;
- **core point:** $|N_\varepsilon(p)| \geq \text{min_samples}$;
- **border point:** inside the ε -neighborhood of a core point but not core itself;
- **noise:** neither core nor border (label -1).

Clusters grow by chaining density-reachable core points, so arbitrary shapes can be discovered and no K is required. A single global ε struggles when densities vary. Because latitude/longitude are spherical coordinates, the **haversine** (great-circle) metric is used with radian inputs; ε is then expressed in radians ($0.03 \text{ rad} \times 6371 \text{ km} \approx 191 \text{ km}$).

Procedure

1. Load the catalogue and filter to magnitude ≥ 4.5 (well-recorded events).
2. Convert latitude/longitude to radians for the haversine metric.
3. Choose ε from the k -distance graph ($k = 10$); the knee is at 0.03 rad.
4. Fit DBSCAN and count clusters and noise events.
5. Show a parameter-sensitivity table for at least two settings.
6. Visualize the epicenters and validate the largest clusters against known seismic regions.

Reference Implementation

```
X_rad = np.radians(quakes[['latitude', 'longitude']].values)
nbrs = NearestNeighbors(n_neighbors=10, metric='haversine').fit(X_rad)
k_distances = np.sort(nbrs.kneighbors(X_rad)[0][:, -1]) # knee ~0.03 rad

dbscan = DBSCAN(eps=0.03, min_samples=10, metric='haversine')
db_labels = dbscan.fit_predict(X_rad)
```

Listing 20: ε from the k -distance knee, then DBSCAN with the haversine metric

Evaluation / Expected Output

Table 8: DBSCAN parameter sensitivity ($\text{min_samples} = 10$).

ε (rad)	approx. km	clusters	noise
0.02	127	70	21.2%
0.03 (selected)	191	55	13.5%
0.05	319	50	5.7%

Table 9: Largest DBSCAN clusters and their dominant region (real-world validation).

Cluster	Events	Center lat/lon	Dominant region
4	1,629	6.1, 126.1	Philippines
0	1,068	−12.5, 160.0	Papua New Guinea
1	947	−24.0, −154.5	Tonga
5	895	30.8, 145.0	Japan region
2	253	−57.8, −26.3	South Sandwich Islands
8	207	37.7, 37.2	Turkey

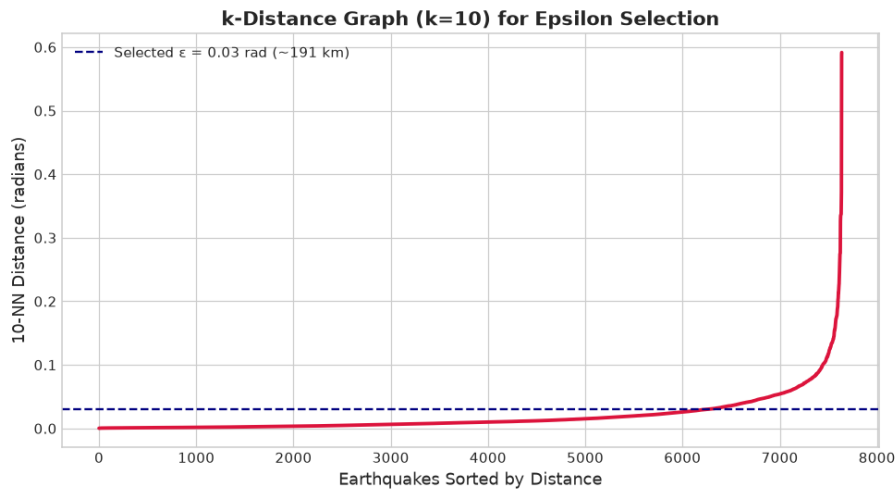


Figure 4: The k -distance graph ($k = 10$); the knee at $\varepsilon = 0.03$ rad marks the transition from dense seismic zones to sparse isolated events.

With $\varepsilon = 0.03$ rad the algorithm finds 55 clusters and labels 13.5% of the events as noise. Smaller ε fragments clusters and labels more events as noise; larger ε merges boundaries and reduces noise. The largest clusters correspond exactly to known seismic zones, and the silhouette on non-noise events (3,000-event sample) is 0.326.

Viva / Discussion Questions

Why does DBSCAN not require K ? Clusters are grown from density-connected core points, so their number and shape emerge from the data instead of being fixed in advance.

What does label -1 mean? The point is neither a core nor a border point — it lies in a sparse region and is treated as noise/outlier (here: isolated earthquakes).

Why can DBSCAN struggle with varying densities? A single ε that is small enough for a dense cluster fragments a sparse one, while one large enough for the sparse cluster merges the dense ones.

Why is haversine used instead of Euclidean distance? Latitude/longitude are spherical coordinates; haversine gives true great-circle distances (radian inputs, ε in radians).

How is ε selected? From the knee of the k -distance graph with $k = \text{min_samples}$ (0.03 rad here).

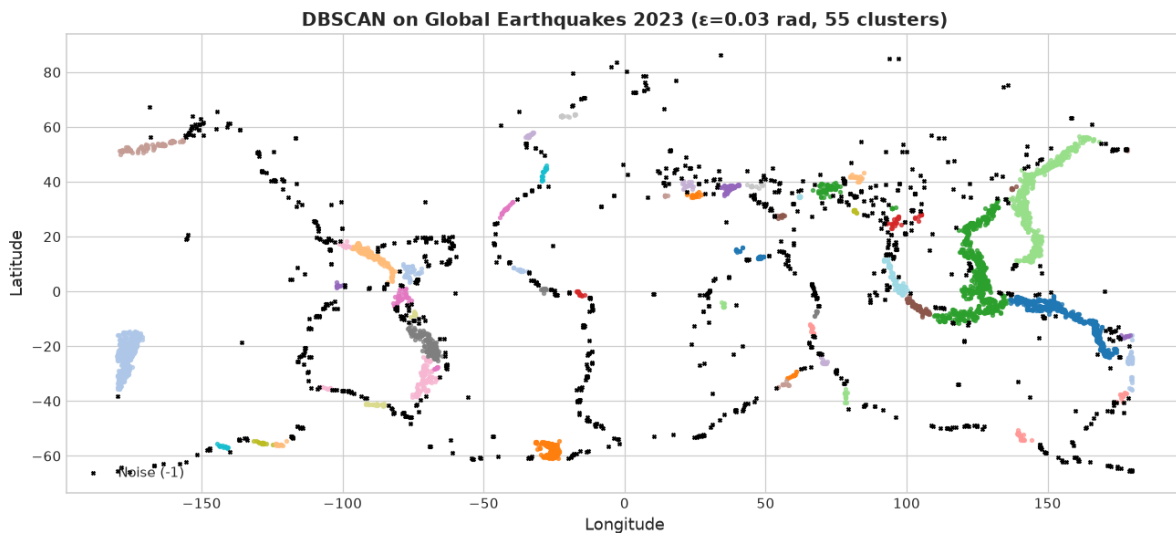


Figure 5: DBSCAN clusters (colors) and noise (black crosses) on the global 2023 earthquake catalogue. The clusters trace the tectonically active plate boundaries.

Complete Code Listing

```
# ---- Core numerical and plotting libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- Density-based clustering algorithms ----
from sklearn.cluster import DBSCAN, HDBSCAN
# ---- Nearest-neighbor search (used to pick epsilon) ----
from sklearn.neighbors import NearestNeighbors
# ---- Evaluation metrics ----
from sklearn.metrics import silhouette_score

# Aesthetics setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (11, 6)
plt.rcParams['figure.dpi'] = 110

np.random.seed(42)
print("Density-based libraries loaded successfully!")
```

Listing 21: Core numerical and plotting libraries

```
# ---- Load the real USGS earthquake catalogue (global, 2023) ----
df = pd.read_csv('../data/earthquakes.csv')
print(f"Full catalogue (magnitude >= 4.0): {len(df):,} events")
print(df.head(3).to_string(index=False))

# Keep well-recorded moderate+ events: magnitude >= 4.5
quakes = df[df['mag'] >= 4.5].reset_index(drop=True)
print(f"\nAnalysed subset (magnitude >= 4.5): {len(quakes):,} events")

# ---- Convert epicenters to radians [latitude, longitude] ----
# scikit-learn's 'haversine' metric expects radian coordinates and computes
# great-circle distance on the sphere (correct for global data).
X_rad = np.radians(quakes[['latitude', 'longitude']].values)
print("Coordinate range (deg): lat", quakes.latitude.min(), "to", quakes.latitude.max(),
      "| lon", quakes.longitude.min(), "to", quakes.longitude.max())
```

Listing 22: Load the real USGS earthquake catalogue (global, 2023)

```
# ---- Choosing epsilon from the k-distance graph (haversine metric) ----
# Sorted distance to each event's k-th nearest neighbor. The knee marks the
# transition from dense seismic zones to sparse, isolated events.
```

```

min_samples = 10
nbrs = NearestNeighbors(n_neighbors=min_samples, metric='haversine').fit(X_rad)
distances, _ = nbrs.kneighbors(X_rad)
k_distances = np.sort(distances[:, -1]) # radians (0.03 rad = 191 km on Earth)

plt.figure(figsize=(9, 5))
plt.plot(k_distances, color='crimson', linewidth=2.5)
plt.title(f"k-Distance Graph (k={min_samples}) for Epsilon Selection", fontsize=14, fontweight='bold')
plt.xlabel("Earthquakes Sorted by Distance", fontsize=12)
plt.ylabel(f"{min_samples}-NN Distance (radians)", fontsize=12)
plt.axhline(y=0.03, color='navy', linestyle='--', label='Selected  $\epsilon = 0.03$  rad (~191 km)')
plt.legend()
plt.tight_layout()
plt.show()

```

Listing 23: Choosing epsilon from the k-distance graph (haversine metric)

```

# ---- Fit DBSCAN with the chosen epsilon and MinPts ----
# DBSCAN grows clusters from core events, connects them via density-reachability,
# and labels everything else as noise (-1).
eps_selected = 0.03 # radians -> 0.03 * 6371 km ~ 191 km
dbscan = DBSCAN(eps=eps_selected, min_samples=min_samples, metric='haversine')
db_labels = dbscan.fit_predict(X_rad)

# Identify which events are core points (returned by the model) vs. border points
core_samples_mask = np.zeros_like(db_labels, dtype=bool)
core_samples_mask[dbscan.core_sample_indices_] = True

n_clusters_db = len(set(db_labels)) - (1 if -1 in db_labels else 0)
n_noise_db = np.sum(db_labels == -1)
print(f"DBSCAN: {n_clusters_db} clusters, {n_noise_db} noise events ({n_noise_db / len(X_rad) * 100:.1f}%)")

# ---- Visualize epicenters: clustered events in color, noise as black crosses ----
plt.figure(figsize=(12, 5.5))
palette = sns.color_palette("tab20", max(n_clusters_db, 1))
for k in range(n_clusters_db):
    mask = db_labels == k
    plt.scatter(quakes.longitude[mask], quakes.latitude[mask], s=6, color=palette[k % 20], alpha=0.75)

noise_mask = db_labels == -1
plt.scatter(quakes.longitude[noise_mask], quakes.latitude[noise_mask], s=6, color='black', marker='x', label='
    ↳ Noise (-1)')
plt.title(f"DBSCAN on Global Earthquakes 2023  $\epsilon$ ={eps_selected} rad, {n_clusters_db} clusters)", fontsize=13,
    ↳ fontweight='bold')
plt.xlabel("Longitude", fontsize=12)
plt.ylabel("Latitude", fontsize=12)
plt.legend(loc='lower left')
plt.tight_layout()
plt.show()

```

Listing 24: Fit DBSCAN with the chosen epsilon and MinPts

```

# ---- Parameter sensitivity: DBSCAN under several (eps, min_samples) settings ----
# The manual requires at least two settings so the effect of eps can be discussed.
print("eps (rad)    approx km    clusters    noise %")
for eps_test in [0.02, 0.03, 0.05]:
    lbl = DBSCAN(eps=eps_test, min_samples=10, metric='haversine').fit_predict(X_rad)
    n_clusters_test = len(set(lbl)) - (1 if -1 in lbl else 0)
    noise_pct = (lbl == -1).mean() * 100
    print(f"    {eps_test:.2f}          {eps_test * 6371:5.0f}          {n_clusters_test:3d}          {noise_pct:5.1f}%")

```

Listing 25: Parameter sensitivity: DBSCAN under several (eps, min_samples) settings

Experiment 6: HDBSCAN

Dataset: USGS earthquakes 2023 (7,638 events, magnitude ≥ 4.5 ; radians + haversine)

Key parameters: `min_cluster_size = 50`, `min_samples = 10`; no global ε

Headline result: 40 clusters, 21.5% noise; silhouette 0.631 on non-noise events

Objective

Apply hierarchical density-based clustering to discover clusters of different densities and identify noise without selecting a global radius.

Suggested Dataset

The same real earthquake catalogue as Experiment 5 (7,638 events, radians + haversine metric).

Required Libraries

scikit-learn, numpy, pandas, matplotlib.

Brief Theory

HDBSCAN removes the need for a global ε by building a hierarchy over all density levels:

1. compute the **core distance** $d_{\text{core}}(p)$ — the distance to a point's k -th nearest neighbor;
2. define the **mutual reachability distance** $d_{\text{mreach}}(a, b) = \max\{d_{\text{core}}(a), d_{\text{core}}(b), d(a, b)\}$;
3. build a **minimum spanning tree** over the mutual-reachability graph;
4. convert the MST into a cluster hierarchy and condense it, tracking cluster births and deaths;
5. keep the clusters with the highest **stability** $S(C) = \sum_{p \in C} (\lambda_p - \lambda_{\text{birth}}(C))$.

The result is a flat clustering with per-point membership probabilities and noise labels, with no radius to tune.

Procedure

1. Reuse the prepared radian coordinates.
2. Fit HDBSCAN with `min_cluster_size = 50` and `min_samples = 10`.
3. Count clusters and noise events; inspect the membership probabilities.
4. Visualize the partition.
5. Compare with DBSCAN.
6. Discuss the effect of `min_cluster_size`.

Reference Implementation

```
hdb = HDBSCAN(min_cluster_size=50, min_samples=10, metric='haversine', copy=False)
hdb_labels = hdb.fit_predict(X_rad)
```

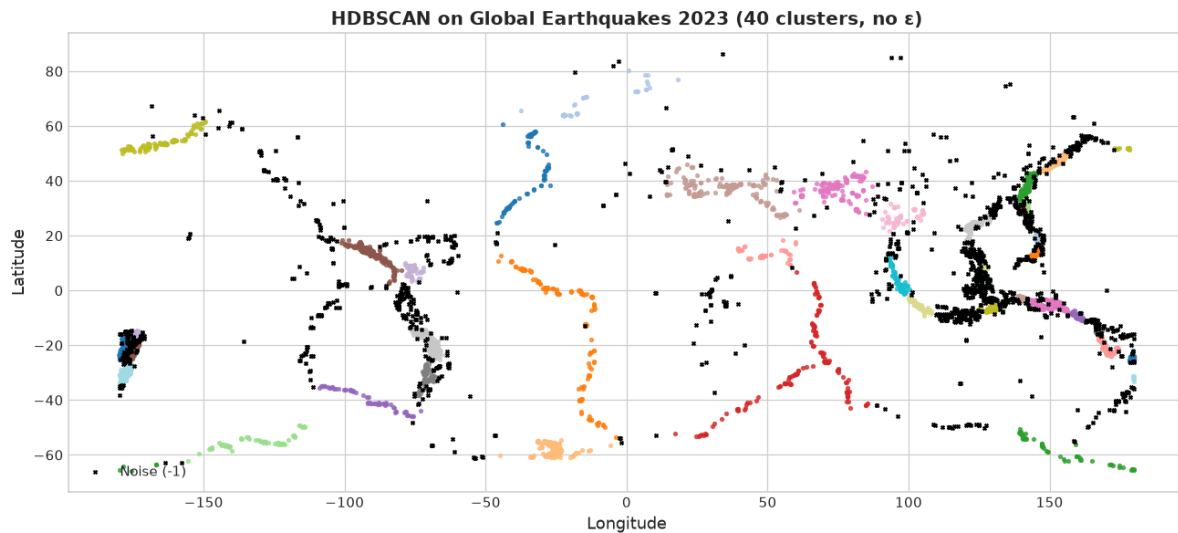
Listing 26: HDBSCAN on the earthquake data (no global epsilon)

Evaluation / Expected Output

HDBSCAN needs no ε and handles the varying density along plate boundaries better than DBSCAN; it is more conservative (21.5% vs. 13.5% noise) because a cluster must remain stable across density scales. Its silhouette is roughly double DBSCAN's because the fixed- ε algorithm chains entire arc-shaped boundaries into single elongated clusters. Membership probabilities are high (> 0.9) deep inside seismic zones and lower near cluster edges.

Table 10: Density-based comparison on the 2023 earthquake catalogue.

Algorithm	Clusters	Noise	Largest cluster	Silhouette
DBSCAN ($\epsilon = 0.03$)	55	13.5%	1,629	0.326
HDBSCAN	40	21.5%	560	0.631

**Figure 6:** HDBSCAN clusters and noise on the same catalogue; adaptive density splits the long DBSCAN arc clusters into compact, well-separated groups.

Viva / Discussion Questions

What is hierarchical about HDBSCAN? It constructs a full hierarchy of density-based clusters across all density levels and then condenses it by tracking cluster birth and death, keeping only the most stable clusters.

What does membership probability mean? A per-point confidence that it belongs to its assigned cluster, derived from the point's persistence as the density threshold changes; low values indicate borderline points.

Why is HDBSCAN useful for variable-density data? Clusters are selected by stability across density levels rather than by one global ϵ , so dense and sparse clusters can coexist.

What does `min_cluster_size` control? The smallest group that may become a cluster; increasing it reduces the number of small clusters, decreasing it fragments the map.

Complete Code Listing

```
# ---- Core numerical and plotting libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- Density-based clustering algorithms ----
from sklearn.cluster import DBSCAN, HDBSCAN
# ---- Nearest-neighbor search (used to pick epsilon) ----
from sklearn.neighbors import NearestNeighbors
# ---- Evaluation metrics ----
from sklearn.metrics import silhouette_score

# Aesthetics setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (11, 6)
plt.rcParams['figure.dpi'] = 110

np.random.seed(42)
print("Density-based libraries loaded successfully!")
```

Listing 27: Core numerical and plotting libraries

```
# ---- Load the real USGS earthquake catalogue (global, 2023) ----
df = pd.read_csv('../data/earthquakes.csv')
print(f"Full catalogue (magnitude >= 4.0): {len(df):,} events")
print(df.head(3).to_string(index=False))

# Keep well-recorded moderate+ events: magnitude >= 4.5
quakes = df[df['mag'] >= 4.5].reset_index(drop=True)
print(f"\nAnalysed subset (magnitude >= 4.5): {len(quakes):,} events")

# ---- Convert epicenters to radians [latitude, longitude] ----
# scikit-learn's 'haversine' metric expects radian coordinates and computes
# great-circle distance on the sphere (correct for global data).
X_rad = np.radians(quakes[['latitude', 'longitude']].values)
print("Coordinate range (deg): lat", quakes.latitude.min(), "to", quakes.latitude.max(),
      "| lon", quakes.longitude.min(), "to", quakes.longitude.max())
```

Listing 28: Load the real USGS earthquake catalogue (global, 2023)

```
# ---- Fit HDBSCAN (no global epsilon required) ----
# min_cluster_size = smallest group considered a cluster; min_samples controls conservativeness.
hdb = HDBSCAN(min_cluster_size=50, min_samples=10, metric='haversine', copy=False)
hdb_labels = hdb.fit_predict(X_rad)

n_clusters_hdb = len(set(hdb_labels)) - (1 if -1 in hdb_labels else 0)
n_noise_hdb = np.sum(hdb_labels == -1)
print(f"HDBSCAN: {n_clusters_hdb} clusters, {n_noise_hdb:,} noise events ({n_noise_hdb / len(X_rad) * 100:.1f}%)")

# ---- Visualize the HDBSCAN partition ----
plt.figure(figsize=(12, 5.5))
palette = sns.color_palette("tab20", max(n_clusters_hdb, 1))
for k in range(n_clusters_hdb):
    mask = hdb_labels == k
    plt.scatter(quakes.longitude[mask], quakes.latitude[mask], s=6, color=palette[k % 20], alpha=0.75)

noise_mask = hdb_labels == -1
plt.scatter(quakes.longitude[noise_mask], quakes.latitude[noise_mask], s=6, color='black', marker='x', label='
↳ Noise (-1)')
plt.title(f"HDBSCAN on Global Earthquakes 2023 ({n_clusters_hdb} clusters, no ε)", fontsize=13, fontweight='bold')
plt.xlabel("Longitude", fontsize=12)
plt.ylabel("Latitude", fontsize=12)
plt.legend(loc='lower left')
plt.tight_layout()
plt.show()
```

Listing 29: Fit HDBSCAN (no global epsilon required)

Part III

Semi-Supervised Learning

Experiment 7: Self-training for Semi-supervised Learning

Dataset: Breast Cancer Wisconsin (569×30 , binary); training set 398 samples with only 15% labeled (61), 337 unlabeled

Key parameters: RBF-kernel SVC ($C = 1$, `probability = True`), confidence threshold $\tau = 0.80$, `max_iter = 15`

Headline result: Baseline 0.9298 $\rightarrow$ self-training 0.9415 (fully supervised ceiling 0.9766)

Objective

Use a small labeled subset and a larger unlabeled subset, iteratively adding high-confidence pseudo-labels, and compare against a supervised baseline and the fully supervised ceiling.

Suggested Dataset

Breast Cancer Wisconsin (569×30 , binary). Stratified 70/30 split; only 15% of the training labels are kept, the rest are masked as unlabeled (-1).

Required Libraries

scikit-learn, numpy.

Brief Theory

Given a labeled set L and an unlabeled pool U , self-training repeats:

1. train the base classifier f on L ;
2. compute class probabilities $f(x)$ for every $x \in U$;
3. select the predictions whose maximum probability exceeds the threshold τ and add them to L as **pseudo-labels**;
4. remove those samples from U ;
5. repeat until no new labels pass the threshold or the iteration limit is reached.

The threshold controls the purity/quantity trade-off: a low τ admits wrong pseudo-labels (confirmation bias), while a very high τ admits too few to help. The base estimator must expose `predict_proba`, which is why the SVC is calibrated (`probability=True`).

Procedure

1. Split the data stratified and scale the features (fit on training data only).
2. Mask 85% of the training labels to -1 .
3. Train a supervised baseline SVC on the labeled 61 samples.
4. Wrap an RBF SVC in `SelfTrainingClassifier` with $\tau = 0.80$ and fit on the labeled plus unlabeled data; monitor the iterations.
5. Train a fully supervised SVC on all 398 labels as the ceiling.
6. Compare accuracy and F1 on the untouched test set.

Reference Implementation

```
self_training_svc = SelfTrainingClassifier(  
    estimator=SVC(kernel='rbf', probability=True, C=1.0, random_state=42),  
    threshold=0.80, criterion='threshold', max_iter=15, verbose=True)  
self_training_svc.fit(X_train_full_scaled, y_train_semi)  # labels + unlabeled (-1)
```

Listing 30: Self-training with an RBF-kernel SVC

Evaluation / Expected Output

Table 11: Self-training compared with the supervised baseline and the fully supervised ceiling.

Setting	Accuracy	F1	Training samples
Supervised baseline (15% labeled)	0.9298	0.9469	61
Self-training (semi-supervised)	0.9415	0.9554	382
Fully supervised ceiling (100%)	0.9766	0.9813	398

Pseudo-labels added per iteration: +292, +22, +5, +2, then termination `no_change` (321 new labels in total). The decaying additions indicate stable convergence rather than runaway error accumulation. Using no extra human labels, self-training improves accuracy by +1.2 points over the baseline, closing part of the gap to the fully supervised model; the remaining gap is the cost of noisy pseudo-labels.

Viva / Discussion Questions

What is a pseudo-label? A predicted label for an unlabeled sample that the model accepts as a training target because its confidence exceeds the threshold.

What is confirmation bias in self-training? The model reinforces its own mistakes: wrong confident predictions enter the training set, make the model more confident about similar errors and can degrade performance. High thresholds and convergence monitoring mitigate this.

Why must the test labels remain hidden during training? Otherwise the evaluation is contaminated: test data would influence the model and the reported accuracy would no longer estimate generalization.

What does the confidence threshold control? The purity/quantity trade-off of pseudo-labels; 0.80 balanced the two here (convergence after four useful iterations).

Why does the SVC need `probability=True`? The wrapper requires `predict_proba`; a plain SVC outputs only decision values, so Platt scaling is enabled.

Complete Code Listing

```
# ---- Suppress a known sklearn deprecation notice for SVC(probability=True) ----
import warnings
# ---- Core numerical and plotting libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ---- Dataset + train/test utilities ----
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
# ---- Semi-supervised self-training wrapper ----
from sklearn.semi_supervised import SelfTrainingClassifier
# ---- Base estimators compared inside self-training ----
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
# ---- Evaluation metrics ----
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, roc_auc_score,
    ↪ confusion_matrix, classification_report

# Aesthetics setup (consistent look across all notebooks)
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110

# The installed scikit-learn still supports SVC(probability=True) but warns; silence that specific notice
warnings.filterwarnings("ignore", message="The `probability` parameter was deprecated", category=FutureWarning)
np.random.seed(42)
print("Semi-supervised libraries imported successfully!")

# ---- Load the Breast Cancer Wisconsin dataset ----
```

```

data = load_breast_cancer()
X_full = data.data          # 30 continuous diagnostic features
y_full = data.target        # 0 = malignant, 1 = benign
feature_names = data.feature_names
target_names = data.target_names

# ---- Train / Test split (stratified to preserve class balance) ----
X_train_full, X_test, y_train_full, y_test = train_test_split(
    X_full, y_full, test_size=0.30, random_state=42, stratify=y_full
)

# ---- Standardize features (fit on train only to avoid test leakage) ----
scaler = StandardScaler()
X_train_full_scaled = scaler.fit_transform(X_train_full)
X_test_scaled = scaler.transform(X_test)

# ---- Simulate the semi-supervised setting: hide 85% of the training labels ----
unlabeled_ratio = 0.85
rng = np.random.RandomState(42)
random_unlabeled_points = rng.rand(len(y_train_full)) < unlabeled_ratio # True -> mask

y_train_semi = np.copy(y_train_full)
y_train_semi[random_unlabeled_points] = -1 # -1 is the scikit-learn convention for "unlabeled"

n_labeled = np.sum(y_train_semi != -1)
n_unlabeled = np.sum(y_train_semi == -1)

print(f"Total Training Samples: {len(y_train_full)}")
print(f" - Labeled Samples (15%): {n_labeled}")
print(f" - Unlabeled Samples (85%): {n_unlabeled}")
print(f"Holdout Test Samples: {len(y_test)}")

```

Listing 31: Suppress a known sklearn deprecation notice for SVC(probability=True)

```

# ---- Experiment 1: Self-Training with an RBF-kernel SVC ----

# 1. Supervised baseline trained ONLY on the ~60 labeled samples
mask_labeled = (y_train_semi != -1) # boolean mask of the labeled subset
X_labeled = X_train_full_scaled[mask_labeled]
y_labeled = y_train_semi[mask_labeled]

base_svc = SVC(kernel='rbf', probability=True, C=1.0, random_state=42)
base_svc.fit(X_labeled, y_labeled)
y_pred_baseline_svc = base_svc.predict(X_test_scaled)
acc_baseline_svc = accuracy_score(y_test, y_pred_baseline_svc)
f1_baseline_svc = f1_score(y_test, y_pred_baseline_svc)

# 2. Semi-supervised self-training: base SVC + iterative pseudo-labeling above threshold 0.80
self_training_svc = SelfTrainingClassifier(
    estimator=SVC(kernel='rbf', probability=True, C=1.0, random_state=42),
    threshold=0.80, # only accept pseudo-labels with confidence >= 0.80
    criterion='threshold', # use the confidence-threshold selection rule
    max_iter=15,
    verbose=True # print how many labels are added per iteration
)
self_training_svc.fit(X_train_full_scaled, y_train_semi)
y_pred_semi_svc = self_training_svc.predict(X_test_scaled)
acc_semi_svc = accuracy_score(y_test, y_pred_semi_svc)
f1_semi_svc = f1_score(y_test, y_pred_semi_svc)

# 3. Fully supervised "ceiling": train on 100% of the ground-truth labels
full_svc = SVC(kernel='rbf', probability=True, C=1.0, random_state=42)
full_svc.fit(X_train_full_scaled, y_train_full)
y_pred_full_svc = full_svc.predict(X_test_scaled)
acc_full_svc = accuracy_score(y_test, y_pred_full_svc)
f1_full_svc = f1_score(y_test, y_pred_full_svc)

# ---- Compare the three settings ----
print(f"\n--- SVC Evaluation Results ---")
print(f"Supervised Baseline (15% labeled): Accuracy = {acc_baseline_svc:.4f}, F1 = {f1_baseline_svc:.4f}")
print(f"Self-Training (Semi-supervised): Accuracy = {acc_semi_svc:.4f}, F1 = {f1_semi_svc:.4f}")
print(f"Fully Supervised Ceiling (100%): Accuracy = {acc_full_svc:.4f}, F1 = {f1_full_svc:.4f}")

```

Listing 32: Experiment 1: Self-Training with an RBF-kernel SVC

```
# ---- Inspect how self-training converged ----
# n_iter_ : number of self-training iterations performed
# termination_condition_ : why training stopped (e.g. 'no_change' = no new confident labels)
# transduction_ : final labels for every training sample (-1 = still unlabeled)
n_iter = self_training_svc.n_iter_
termination_condition = self_training_svc.termination_condition_
labeled_final_mask = (self_training_svc.transduction_ != -1)

print(f"Self-Training Convergence Summary:")
print(f" - Iterations completed: {n_iter}")
print(f" - Termination condition: {termination_condition}")
print(f" - Total samples labeled after self-training: {np.sum(labeled_final_mask)} / {len(y_train_full)}")
print(f" - Pseudo-labels newly added: {np.sum(labeled_final_mask) - n_labeled}")
```

Listing 33: Inspect how self-training converged

```
# ---- Compact comparison table (SVC) ----
summary_data = [
    {"Model": "SVC (Baseline 15% Labeled)", "Accuracy": acc_baseline_svc, "F1-Score": f1_baseline_svc,
     "Training Samples": n_labeled},
    {"Model": "SVC (Self-Training Semi-Supervised)", "Accuracy": acc_semi_svc, "F1-Score": f1_semi_svc,
     "Training Samples": np.sum(self_training_svc.transduction_ != -1)},
    {"Model": "SVC (Fully Supervised Ceiling 100%)", "Accuracy": acc_full_svc, "F1-Score": f1_full_svc,
     "Training Samples": len(y_train_full)},
]
summary_df = pd.DataFrame(summary_data).set_index("Model")
display(summary_df.style.highlight_max(subset=['Accuracy', 'F1-Score'], color='lightgreen'))
```

Listing 34: Compact comparison table (SVC)

Part IV

Ensemble Learning

Experiment 8: Random Forest Regression (RFR)

Dataset: California Housing, 5,000-row random sample, 8 features; 80/20 split
Key parameters: 150 trees, max_depth = 12, out-of-bag scoring enabled
Headline result: Test R^2 0.7415 (OOB 0.7513); RMSE 0.5908; MAE 0.3999

Objective

Predict a continuous target using an ensemble of randomized decision trees; evaluate with MAE, RMSE and R^2 and inspect out-of-bag validation.

Suggested Dataset

California Housing (scikit-learn), 5,000-row random sample, target = median house value.

Required Libraries

scikit-learn, numpy, pandas.

Brief Theory

Random Forest Regression trains B decision trees on bootstrap samples of the training data; at each split only a random subset of features is considered, which decorrelates the trees. The ensemble prediction is the average of the tree outputs,

$$\hat{f}_{\text{RF}}(x) = \frac{1}{B} \sum_{b=1}^B T_b(x).$$

Averaging approximately uncorrelated trees reduces variance: for pairwise correlation ρ , $\text{Var}(\hat{f}_{\text{RF}}) = \rho\sigma^2 + \frac{1-\rho}{B}\sigma^2$, so adding trees can only help. Out-of-bag (OOB) samples — the $\approx 37\%$ not drawn in a bootstrap — provide a free internal validation estimate.

Procedure

1. Load and sample the dataset; split into training and test sets.
2. Train `RandomForestRegressor` with 150 trees, max_depth = 12 and OOB scoring enabled.
3. Predict the test set and compute R^2 , RMSE and MAE.
4. Report the OOB R^2 and compare it with the test R^2 to check overfitting.
5. Inspect feature importances.

Reference Implementation

```
rf_reg = RandomForestRegressor(n_estimators=150, max_depth=12,
                              oob_score=True, random_state=42, n_jobs=-1)
rf_reg.fit(X_train_reg, y_train_reg)
print(rf_reg.oob_score_, r2_score(y_test_reg, rf_reg.predict(X_test_reg)))
```

Listing 35: Random Forest Regression with out-of-bag validation

Evaluation / Expected Output

The OOB R^2 (0.7513) is very close to the test R^2 (0.7415), so the model is not overfitting and the OOB estimate is a reliable free validation. Feature importances are dominated by median income and location (latitude/longitude), consistent with housing economics.

Table 12: Random Forest Regression on the California Housing sample.

Metric	Value
OOB R^2	0.7513
Test R^2	0.7415
Test RMSE	0.5908 (in \$100k units $\approx$ \$59k)
Test MAE	0.3999 (in \$100k units $\approx$ \$40k)

Viva / Discussion Questions

Why does a forest reduce variance? Averaging B approximately uncorrelated trees gives $\rho\sigma^2 + (1 - \rho)\sigma^2/B$; as B grows the second term vanishes, so predictions are more stable than a single tree's.

What is bagging? Bootstrap aggregating: each model is trained on a bootstrap sample of the data and the predictions are combined (average for regression, vote for classification).

Why can feature importance be misleading? Impurity-based importance is biased toward high-cardinality/continuous features, is computed on training data (it implies no causality), and correlated features share or dilute importance arbitrarily.

Complete Code Listing

```
# ---- Core libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier, AdaBoostRegressor, AdaBoostClassifier
from sklearn.metrics import r2_score, root_mean_squared_error, mean_absolute_error, accuracy_score, f1_score,
    ↪ roc_auc_score

import xgboost as xgb
import catboost as cb

# Styling setup
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110
np.random.seed(42)
```

Listing 36: Core libraries

```
# ---- Load the California Housing dataset (real-world regression benchmark) ----
housing = fetch_california_housing(as_frame=True)
# Sample 5,000 rows so every ensemble trains quickly while remaining representative
df_reg = housing.frame.sample(n=5000, random_state=42)

X_reg = df_reg[housing.feature_names] # 8 socio-economic / geographic features
y_reg = df_reg['MedHouseVal']         # target: median house value

# ---- Train/test split (80% / 20%) ----
X_train_reg, X_test_reg, y_train_reg, y_test_reg = train_test_split(
    X_reg, y_reg, test_size=0.20, random_state=42
)

print(f"Regression Training Samples: {X_train_reg.shape[0]}, Test Samples: {X_test_reg.shape[0]}")
df_reg.head()
```

Listing 37: Load the California Housing dataset (real-world regression benchmark)

```
# ---- Random Forest Regression (bagging of decision trees) ----
# n_estimators = number of trees, oob_score = estimate accuracy from out-of-bag samples
rf_reg = RandomForestRegressor(n_estimators=150, max_depth=12, oob_score=True, random_state=42, n_jobs=-1)
rf_reg.fit(X_train_reg, y_train_reg)
```

```
# ---- Predict and evaluate ----
y_pred_rf = rf_reg.predict(X_test_reg)
r2_rf = r2_score(y_test_reg, y_pred_rf)           # explained variance (1.0 = perfect)
rmse_rf = root_mean_squared_error(y_test_reg, y_pred_rf) # error in target units
mae_rf = mean_absolute_error(y_test_reg, y_pred_rf)    # robust average error

print(f"Random Forest Regressor Results:")
print(f" - Out-of-Bag (OOB) R2 Score: {rf_reg.oob_score_:.4f}")
print(f" - Test R2 Score: {r2_rf:.4f}")
print(f" - Test RMSE: {rmse_rf:.4f}")
print(f" - Test MAE: {mae_rf:.4f}")
```

Listing 38: Random Forest Regression (bagging of decision trees)

Experiment 9: Random Forest Classification (RFC)

Dataset: Heart Disease (`data/heart_disease.csv`, 303 rows, 13 features); stratified 75/25 split
Key parameters: 120 trees, `max_depth = 8`; one-hot encoded categorical features
Headline result: Accuracy 0.7763; F1 0.8046; ROC-AUC 0.8704

Objective

Classify observations using an ensemble of randomized decision trees, with a stratified split and classification metrics.

Suggested Dataset

Heart Disease (`data/heart_disease.csv`), 303 rows, 13 features plus the binary target.

Required Libraries

scikit-learn, numpy, pandas.

Brief Theory

Random Forest Classification aggregates tree votes. Each tree votes for one class and the ensemble predicts the most frequent class,

$$\hat{C}_{\text{RF}}(x) = \arg \max_k \sum_{b=1}^B \mathbb{I}(T_b(x) = k).$$

Bootstrap sampling plus feature subsampling decorrelates the trees, reducing variance relative to a single decision tree. The fraction of trees voting for the positive class also provides a class probability, which is used for ROC-AUC.

Procedure

1. Load the heart data; one-hot encode the categorical features.
2. Use a stratified 75/25 split to preserve the disease ratio.
3. Train `RandomForestClassifier` with 120 trees and `max_depth = 8`.
4. Predict labels and probabilities on the test set.
5. Compute accuracy, F1 and ROC-AUC.

Reference Implementation

```
rfc = RandomForestClassifier(n_estimators=120, max_depth=8, random_state=42, n_jobs=-1)
rfc.fit(X_train_clf, y_train_clf)
y_prob_rfc = rfc.predict_proba(X_test_clf)[:, 1]
```

Listing 39: Random Forest Classification with stratified evaluation

Evaluation / Expected Output

Table 13: Random Forest Classification on Heart Disease (76 test samples).

Metric	Value
Accuracy	0.7763
F1-score	0.8046
ROC-AUC	0.8704

ROC-AUC 0.87 indicates good ranking quality on a small dataset; accuracy 0.776 is typical for heart-disease prediction with these features and reflects the problem's noise rather than a modelling failure. F1 and AUC are safer summaries than accuracy alone because the classes are not perfectly balanced.

Viva / Discussion Questions

RFR vs RFC? Regression averages the trees' numeric outputs; classification takes a majority vote (and averages class probabilities for scoring).

What is majority voting? Each tree votes for one class and the most frequent class becomes the ensemble prediction; soft voting averages the probabilities instead.

How can class imbalance affect accuracy? With a rare positive class, a model can predict the majority class everywhere and still show high accuracy; precision, recall, F1 and AUC expose the failure.

Complete Code Listing

```
# ---- Core libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier, AdaBoostRegressor, AdaBoostClassifier
from sklearn.metrics import r2_score, root_mean_squared_error, mean_absolute_error, accuracy_score, f1_score,
    ↪ roc_auc_score

import xgboost as xgb
import catboost as cb

# Styling setup
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110
np.random.seed(42)
```

Listing 40: Core libraries

```
# ---- Load the Heart Disease dataset (real-world classification benchmark) ----
df_clf = pd.read_csv('../data/heart_disease.csv')
print(f"Heart Disease dataset: {df_clf.shape[0]} rows x {df_clf.shape[1]} columns")

# Separate target and features (handle either 'target' or 'Target' as the column name)
target_col = 'target' if 'target' in df_clf.columns else 'Target'
y_clf = df_clf[target_col].values
X_clf_raw = df_clf.drop(columns=[target_col])

# One-hot encode categorical variables for RFC / XGBoost / AdaBoost
X_clf_encoded = pd.get_dummies(X_clf_raw, drop_first=True)

# Stratified split keeps the disease/no-disease ratio identical in train and test
X_train_clf, X_test_clf, y_train_clf, y_test_clf = train_test_split(
    X_clf_encoded, y_clf, test_size=0.25, random_state=42, stratify=y_clf
)

print(f"Classification Training Set: {X_train_clf.shape}, Test Set: {X_test_clf.shape}")

# ---- Random Forest Classification (majority vote of trees) ----
rfc = RandomForestClassifier(n_estimators=120, max_depth=8, random_state=42, n_jobs=-1)
rfc.fit(X_train_clf, y_train_clf)

# Predictions and probability of the positive (disease) class
y_pred_rfc = rfc.predict(X_test_clf)
y_prob_rfc = rfc.predict_proba(X_test_clf)[:, 1]

# ---- Evaluate: accuracy, F1 and ROC-AUC ----
acc_rfc = accuracy_score(y_test_clf, y_pred_rfc)
f1_rfc = f1_score(y_test_clf, y_pred_rfc)
```

```
auc_rfc = roc_auc_score(y_test_clf, y_prob_rfc)

print(f"Random Forest Classifier: Accuracy = {acc_rfc:.4f}, F1 = {f1_rfc:.4f}, ROC-AUC = {auc_rfc:.4f}")
```

Listing 41: Load the Heart Disease dataset (real-world classification benchmark)

Experiment 10: XGBoost

Dataset: Heart Disease (same stratified 75/25 split as RFC)

Key parameters: 120 trees, learning rate 0.08, max_depth = 4, subsample / colsample = 0.8

Headline result: Accuracy 0.7763; F1 0.8132; ROC-AUC 0.8509

Objective

Train a gradient-boosted tree classifier and analyze the effect of boosting hyperparameters.

Suggested Dataset

Heart Disease (data/heart_disease.csv), the same stratified split used for the other ensembles.

Required Libraries

xgboost, scikit-learn, numpy, pandas.

Brief Theory

XGBoost builds trees sequentially, each fitting a second-order Taylor expansion of the loss:

$$\mathcal{L}^{(t)} \approx \sum_{i=1}^n \left[g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t), \quad \Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2,$$

where g_i and h_i are the first and second derivatives of the loss with respect to the current prediction. The regularization term Ω penalizes tree complexity and leaf weights, while row/column subsampling adds stochasticity; both control overfitting.

Procedure

1. Prepare the one-hot encoded heart data and the same stratified split.
2. Fit `XGBClassifier` with 120 trees, learning rate 0.08, depth 4, subsample and column sample 0.8, and the logistic-loss objective.
3. Predict labels and probabilities on the test set.
4. Report accuracy, F1 and ROC-AUC.
5. Discuss the complexity/generalization trade-off.

Reference Implementation

```
xgb_clf = xgb.XGBClassifier(n_estimators=120, learning_rate=0.08, max_depth=4,
                           subsample=0.8, colsample_bytree=0.8,
                           random_state=42, eval_metric='logloss')
xgb_clf.fit(X_train_clf, y_train_clf)
y_prob_xgb_c = xgb_clf.predict_proba(X_test_clf)[:, 1]
```

Listing 42: XGBoost classification on the heart data

Evaluation / Expected Output

XGBoost matches RFC on accuracy with a slightly higher F1 but a slightly lower AUC on this small test set; differences of 0.01–0.02 are within noise for 76 samples. On the larger regression benchmark, gradient boosting clearly leads (Experiment 8 comparison), which is the usual pattern for tabular data.

Table 14: XGBoost classification on Heart Disease (76 test samples).

Metric	Value
Accuracy	0.7763
F1-score	0.8132
ROC-AUC	0.8509

Viva / Discussion Questions

Bagging vs boosting? Bagging trains models in parallel on bootstrap samples and combines them to reduce variance; boosting trains models sequentially so each corrects the previous ensemble's errors, mainly reducing bias.

What does the learning rate do? It shrinks each tree's contribution; smaller values need more trees but usually generalize better, larger values fit faster and can overfit.

Why can boosting overfit? Later trees keep fitting remaining errors, including label noise, so the ensemble can become too complex; depth limits, regularization, subsampling and early stopping control this.

Complete Code Listing

```
# ---- Core libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier, AdaBoostRegressor, AdaBoostClassifier
from sklearn.metrics import r2_score, root_mean_squared_error, mean_absolute_error, accuracy_score, f1_score,
    ↪ roc_auc_score

import xgboost as xgb
import catboost as cb

# Styling setup
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110
np.random.seed(42)
```

Listing 43: Core libraries

```
# ---- Load the Heart Disease dataset (real-world classification benchmark) ----
df_clf = pd.read_csv('../data/heart_disease.csv')
print(f"Heart Disease dataset: {df_clf.shape[0]} rows x {df_clf.shape[1]} columns")

# Separate target and features (handle either 'target' or 'Target' as the column name)
target_col = 'target' if 'target' in df_clf.columns else 'Target'
y_clf = df_clf[target_col].values
X_clf_raw = df_clf.drop(columns=[target_col])

# One-hot encode categorical variables for RFC / XGBoost / AdaBoost
X_clf_encoded = pd.get_dummies(X_clf_raw, drop_first=True)

# Stratified split keeps the disease/no-disease ratio identical in train and test
X_train_clf, X_test_clf, y_train_clf, y_test_clf = train_test_split(
    X_clf_encoded, y_clf, test_size=0.25, random_state=42, stratify=y_clf
)

print(f"Classification Training Set: {X_train_clf.shape}, Test Set: {X_test_clf.shape}")

# ---- Random Forest Classification (majority vote of trees) ----
rfc = RandomForestClassifier(n_estimators=120, max_depth=8, random_state=42, n_jobs=-1)
rfc.fit(X_train_clf, y_train_clf)

# Predictions and probability of the positive (disease) class
y_pred_rfc = rfc.predict(X_test_clf)
```

```
y_prob_rfc = rfc.predict_proba(X_test_clf)[:, 1]

# ---- Evaluate: accuracy, F1 and ROC-AUC ----
acc_rfc = accuracy_score(y_test_clf, y_pred_rfc)
f1_rfc = f1_score(y_test_clf, y_pred_rfc)
auc_rfc = roc_auc_score(y_test_clf, y_prob_rfc)

print(f"Random Forest Classifier: Accuracy = {acc_rfc:.4f}, F1 = {f1_rfc:.4f}, ROC-AUC = {auc_rfc:.4f}")
```

Listing 44: Load the Heart Disease dataset (real-world classification benchmark)

```
# ---- XGBoost classification on the same stratified split ----
xgb_clf = xgb.XGBClassifier(
    n_estimators=120, learning_rate=0.08, max_depth=4,
    subsample=0.8, colsample_bytree=0.8,
    random_state=42, eval_metric='logloss'
)
xgb_clf.fit(X_train_clf, y_train_clf)

y_pred_xgb_c = xgb_clf.predict(X_test_clf)
y_prob_xgb_c = xgb_clf.predict_proba(X_test_clf)[:, 1]

acc_xgb_c = accuracy_score(y_test_clf, y_pred_xgb_c)
f1_xgb_c = f1_score(y_test_clf, y_pred_xgb_c)
auc_xgb_c = roc_auc_score(y_test_clf, y_prob_xgb_c)
print(f"XGBoost Classifier: Accuracy = {acc_xgb_c:.4f}, F1 = {f1_xgb_c:.4f}, ROC-AUC = {auc_xgb_c:.4f}")
```

Listing 45: XGBoost classification on the same stratified split

Experiment 11: AdaBoost

Dataset: Heart Disease (same stratified 75/25 split)

Key parameters: 100 decision stumps, learning rate 0.1

Headline result: Accuracy 0.7895; F1 0.8182; ROC-AUC 0.8714 (best of the four ensembles)

Objective

Build an adaptive boosting classifier and study how weak learners combine into a stronger ensemble.

Suggested Dataset

Heart Disease (`data/heart_disease.csv`), same split as the other ensembles.

Required Libraries

scikit-learn, numpy, pandas.

Brief Theory

AdaBoost trains weak learners (decision stumps) sequentially on re-weighted data. Misclassified samples receive larger weights, so each new learner focuses on the current errors. At round m the weighted error ϵ_m determines the learner weight

$$\alpha_m = \frac{1}{2} \ln \left(\frac{1 - \epsilon_m}{\epsilon_m} \right),$$

and the sample weights are updated as $w_i^{(m+1)} = w_i^{(m)} \exp(-\alpha_m y_i h_m(x_i))$. The final prediction is a weighted vote of all stumps. Because the exponential loss grows quickly for hard points, AdaBoost is sensitive to label noise and outliers.

Procedure

1. Prepare the heart data and the stratified split.
2. Fit `AdaBoostClassifier` with 100 stumps and learning rate 0.1.
3. Predict labels and probabilities on the test set.
4. Report accuracy, F1 and ROC-AUC; compare with RFC and XGBoost.

Reference Implementation

```
ada_clf = AdaBoostClassifier(n_estimators=100, learning_rate=0.1, random_state=42)
ada_clf.fit(X_train_clf, y_train_clf)
y_prob_ada_c = ada_clf.predict_proba(X_test_clf)[:, 1]
```

Listing 46: AdaBoost classification

Evaluation / Expected Output

Table 15: AdaBoost classification on Heart Disease (76 test samples).

Metric	Value
Accuracy	0.7895
F1-score	0.8182
ROC-AUC	0.8714

AdaBoost with shallow stumps slightly outperforms the deeper models on this small dataset — a common outcome

when the signal is mostly additive and the sample size is limited. It remains a cheap, strong baseline; its weakness is sensitivity to label noise, where regularized gradient boosting is more robust.

Viva / Discussion Questions

Why use weak learners? Individually they are simple and low-variance; boosting combines many of them so the ensemble can fit complex boundaries while each step stays simple.

How does AdaBoost focus on difficult examples? After each round, weights of misclassified samples increase, so the next learner optimizes a distribution that emphasizes current errors.

What is the effect of noisy labels? Mislabeled points look “hard”, receive ever-larger weights and can dominate training, degrading accuracy — AdaBoost is known to be noise-sensitive.

Complete Code Listing

```
# ---- Core libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier, AdaBoostRegressor, AdaBoostClassifier
from sklearn.metrics import r2_score, root_mean_squared_error, mean_absolute_error, accuracy_score, f1_score,
    ↪ roc_auc_score

import xgboost as xgb
import catboost as cb

# Styling setup
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110
np.random.seed(42)
```

Listing 47: Core libraries

```
# ---- Load the Heart Disease dataset (real-world classification benchmark) ----
df_clf = pd.read_csv('../data/heart_disease.csv')
print(f"Heart Disease dataset: {df_clf.shape[0]} rows x {df_clf.shape[1]} columns")

# Separate target and features (handle either 'target' or 'Target' as the column name)
target_col = 'target' if 'target' in df_clf.columns else 'Target'
y_clf = df_clf[target_col].values
X_clf_raw = df_clf.drop(columns=[target_col])

# One-hot encode categorical variables for RFC / XGBoost / AdaBoost
X_clf_encoded = pd.get_dummies(X_clf_raw, drop_first=True)

# Stratified split keeps the disease/no-disease ratio identical in train and test
X_train_clf, X_test_clf, y_train_clf, y_test_clf = train_test_split(
    X_clf_encoded, y_clf, test_size=0.25, random_state=42, stratify=y_clf
)

print(f"Classification Training Set: {X_train_clf.shape}, Test Set: {X_test_clf.shape}")

# ---- Random Forest Classification (majority vote of trees) ----
rfc = RandomForestClassifier(n_estimators=120, max_depth=8, random_state=42, n_jobs=-1)
rfc.fit(X_train_clf, y_train_clf)

# Predictions and probability of the positive (disease) class
y_pred_rfc = rfc.predict(X_test_clf)
y_prob_rfc = rfc.predict_proba(X_test_clf)[:, 1]

# ---- Evaluate: accuracy, F1 and ROC-AUC ----
acc_rfc = accuracy_score(y_test_clf, y_pred_rfc)
f1_rfc = f1_score(y_test_clf, y_pred_rfc)
auc_rfc = roc_auc_score(y_test_clf, y_prob_rfc)
```

```
print(f"Random Forest Classifier: Accuracy = {acc_rfc:.4f}, F1 = {f1_rfc:.4f}, ROC-AUC = {auc_rfc:.4f}")
```

Listing 48: Load the Heart Disease dataset (real-world classification benchmark)

```
# ---- AdaBoost classification ----
ada_clf = AdaBoostClassifier(n_estimators=100, learning_rate=0.1, random_state=42)
ada_clf.fit(X_train_clf, y_train_clf)

y_pred_ada_c = ada_clf.predict(X_test_clf)
y_prob_ada_c = ada_clf.predict_proba(X_test_clf)[:, 1]

acc_ada_c = accuracy_score(y_test_clf, y_pred_ada_c)
f1_ada_c = f1_score(y_test_clf, y_pred_ada_c)
auc_ada_c = roc_auc_score(y_test_clf, y_prob_ada_c)
print(f"AdaBoost Classifier: Accuracy = {acc_ada_c:.4f}, F1 = {f1_ada_c:.4f}, ROC-AUC = {auc_ada_c:.4f}")
```

Listing 49: AdaBoost classification

Experiment 12: CatBoost

Dataset: Heart Disease (raw table with categorical features: sex, cp, fbs, restecg, exang, slope, ca, thal)

Key parameters: 150 iterations, depth 5, learning rate 0.08, native `cat_features`

Headline result: Accuracy 0.7632; F1 0.7955; ROC-AUC 0.8551

Objective

Train a gradient-boosted decision-tree model with native handling of categorical variables.

Suggested Dataset

Heart Disease (`data/heart_disease.csv`) using the raw (unencoded) table so CatBoost can process categorical columns directly.

Required Libraries

catboost, pandas, scikit-learn.

Brief Theory

CatBoost is a gradient-boosting library designed for tables that mix numerical and categorical features. It uses **ordered boosting** — leaf values are computed on random permutations of the data — to avoid target leakage and prediction shift, and builds **oblivious (symmetric) trees**, where the same split condition is used at every node of a depth level. Categorical features are encoded internally with ordered target statistics, so the categorical columns must be declared through `cat_features` and the target must never leak into the encoding.

Procedure

1. Identify the categorical columns in the raw heart table and cast them to string.
2. Use the same stratified 75/25 split on the raw features.
3. Fit `CatBoostClassifier` with 150 iterations, depth 5, learning rate 0.08 and the categorical columns declared.
4. Predict labels and probabilities; report accuracy, F1 and ROC-AUC.
5. Compare all four classifiers on the same split.

Reference Implementation

```
categorical_cols = X_clf_raw.select_dtypes(include=['object', 'category']).columns.tolist()
cb_clf = cb.CatBoostClassifier(iterations=150, learning_rate=0.08, depth=5,
                              cat_features=categorical_cols, random_seed=42, verbose=False)
cb_clf.fit(X_tr_cb, y_tr_cb)
```

Listing 50: CatBoost with native categorical features

Evaluation / Expected Output

Table 16: Ensemble classification comparison on the same Heart Disease split (76 test samples).

Model	Accuracy	F1	ROC-AUC
Random Forest (RFC)	0.7763	0.8046	0.8704
XGBoost	0.7763	0.8132	0.8509
AdaBoost	0.7895	0.8182	0.8714
CatBoost	0.7632	0.7955	0.8551

CatBoost performs at a similar level to the other ensembles without any manual one-hot encoding — its native categorical handling does the encoding internally. On this tiny dataset the advanced target statistics cannot show their full advantage (they matter most with high-cardinality categories, e.g. thousands of city or user IDs); all differences are within noise for 76 test samples.

Viva / Discussion Questions

How does CatBoost treat categorical features? It computes ordered target statistics (the mean target per category on random permutations, with prior smoothing) rather than one-hot vectors.

Why should categories not be encoded using target leakage? Plain target encoding uses each row's own target when encoding that row, which leaks label information and inflates training performance; ordered statistics avoid this.

CatBoost vs ordinary one-hot encoding? One-hot explodes dimensionality for high-cardinality features and loses category statistics; CatBoost encodes compactly and exploits target–category relationships with leakage control.

Complete Code Listing

```
# ---- Core libraries ----
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier, AdaBoostRegressor, AdaBoostClassifier
from sklearn.metrics import r2_score, root_mean_squared_error, mean_absolute_error, accuracy_score, f1_score,
    ↪ roc_auc_score

import xgboost as xgb
import catboost as cb

# Styling setup
plt.style.use('seaborn-v0_8-whitegrid' if 'seaborn-v0_8-whitegrid' in plt.style.available else 'default')
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'
plt.rcParams['figure.figsize'] = (10, 6)
plt.rcParams['figure.dpi'] = 110
np.random.seed(42)
```

Listing 51: Core libraries

```
# ---- Load the Heart Disease dataset (real-world classification benchmark) ----
df_clf = pd.read_csv('../data/heart_disease.csv')
print(f"Heart Disease dataset: {df_clf.shape[0]} rows x {df_clf.shape[1]} columns")

# Separate target and features (handle either 'target' or 'Target' as the column name)
target_col = 'target' if 'target' in df_clf.columns else 'Target'
y_clf = df_clf[target_col].values
X_clf_raw = df_clf.drop(columns=[target_col])

# One-hot encode categorical variables for RFC / XGBoost / AdaBoost
X_clf_encoded = pd.get_dummies(X_clf_raw, drop_first=True)

# Stratified split keeps the disease/no-disease ratio identical in train and test
X_train_clf, X_test_clf, y_train_clf, y_test_clf = train_test_split(
    X_clf_encoded, y_clf, test_size=0.25, random_state=42, stratify=y_clf
)

print(f"Classification Training Set: {X_train_clf.shape}, Test Set: {X_test_clf.shape}")

# ---- Random Forest Classification (majority vote of trees) ----
rfc = RandomForestClassifier(n_estimators=120, max_depth=8, random_state=42, n_jobs=-1)
rfc.fit(X_train_clf, y_train_clf)

# Predictions and probability of the positive (disease) class
y_pred_rfc = rfc.predict(X_test_clf)
y_prob_rfc = rfc.predict_proba(X_test_clf)[:, 1]

# ---- Evaluate: accuracy, F1 and ROC-AUC ----
acc_rfc = accuracy_score(y_test_clf, y_pred_rfc)
```

```
fl_rfc = fl_score(y_test_clf, y_pred_rfc)
auc_rfc = roc_auc_score(y_test_clf, y_prob_rfc)

print(f"Random Forest Classifier: Accuracy = {acc_rfc:.4f}, F1 = {fl_rfc:.4f}, ROC-AUC = {auc_rfc:.4f}")
```

Listing 52: Load the Heart Disease dataset (real-world classification benchmark)

```
# ---- CatBoost classification with native categorical features ----
# Identify categorical columns in the raw (unencoded) heart data
categorical_cols = X_clf_raw.select_dtypes(include=['object', 'category']).columns.tolist()
X_raw_cb = X_clf_raw.copy()
for col in categorical_cols:
    X_raw_cb[col] = X_raw_cb[col].astype(str)

# Same stratified split on the raw feature table
X_tr_cb, X_te_cb, y_tr_cb, y_te_cb = train_test_split(
    X_raw_cb, y_clf, test_size=0.25, random_state=42, stratify=y_clf
)

cb_clf = cb.CatBoostClassifier(
    iterations=150, learning_rate=0.08, depth=5,
    cat_features=categorical_cols, random_seed=42, verbose=False
)
cb_clf.fit(X_tr_cb, y_tr_cb)

y_pred_cb_c = cb_clf.predict(X_te_cb)
y_prob_cb_c = cb_clf.predict_proba(X_te_cb)[: , 1]

acc_cb_c = accuracy_score(y_te_cb, y_pred_cb_c)
fl_cb_c = fl_score(y_te_cb, y_pred_cb_c)
auc_cb_c = roc_auc_score(y_te_cb, y_prob_cb_c)
print(f"CatBoost Classifier: Accuracy = {acc_cb_c:.4f}, F1 = {fl_cb_c:.4f}, ROC-AUC = {auc_cb_c:.4f}")
```

Listing 53: CatBoost classification with native categorical features

Part V

Neural Networks: Multilayer Perceptron

Experiment 13: Multilayer Perceptron (MLP)

Dataset: Digits (scikit-learn): 1,797 samples, 8×8 images (64 features), 10 classes; 70/15/15 split

Key parameters: scikit-learn (128, 64) ReLU + Adam + early stopping; PyTorch 64–128–64–10 with BatchNorm and Dropout

Headline result: scikit-learn 0.9630; PyTorch 0.9815 test accuracy (GPU)

Objective

Implement feed-forward neural networks for handwritten-digit classification and study hidden-layer size and regularization.

Suggested Dataset

Digits (scikit-learn): 1,797 samples, 8×8 grayscale images (64 features), 10 classes; stratified 70/15/15 train/validation/test split, standardized on the training data only.

Required Libraries

scikit-learn, PyTorch, numpy, pandas, matplotlib.

Content, result plots and complete code listings for this experiment are filled in after the format review.

Brief Theory

Procedure

Reference Implementation

Evaluation / Expected Output

Viva / Discussion Questions

Complete Code Listing

Part VI

Sequence Modeling: Recurrent Neural Networks

Experiment 14: Recurrent Neural Network (RNN)

Dataset: Synthetic 20-step sequences (1,200; label = positive sum) and Monthly Airline Passengers (12-month windows)

Key parameters: Vanilla RNN vs LSTM; 2 layers, hidden 64 (forecasting) / 32 (classification); Adam; MSE / BCE loss

Headline result: Forecasting: LSTM RMSE 44.12 vs RNN 69.93 passengers; Classification: LSTM 0.9450 vs RNN 0.9300

Objective

Build recurrent networks for sequential data — sequence classification and time-series forecasting — and compare a vanilla RNN with an LSTM.

Suggested Dataset

(a) Synthetic sequence benchmark: 1,200 sequences of length 20, label = 1 if the sum is positive (1,000 train / 200 test). (b) Monthly Airline Passengers (`data/airline_passengers.csv`): 144 months, 12-month sliding windows, chronological 80/20 split.

Required Libraries

PyTorch, numpy, pandas, matplotlib (the manual's TensorFlow/Keras example is implemented equivalently in PyTorch, the framework installed in this environment).

Content, result plots and complete code listings for this experiment are filled in after the format review.

Brief Theory

Procedure

Reference Implementation

Evaluation / Expected Output

Viva / Discussion Questions

Complete Code Listing

Part VII

Unsupervised Neural Learning: Self-Organizing Maps

Experiment 15: Self-Organizing Map (SOM)

Dataset: UCI Wine: 178 samples, 13 chemical features, 3 cultivars (labels used only for visualization)

Key parameters: 12×12 grid, Gaussian neighborhood, $\sigma = 1.5$, learning rate 0.5, PCA initialization, 5,000 iterations

Headline result: Quantization error $0.4447 \rightarrow 0.1965$; topographic error 0.0393

Objective

Use a Self-Organizing Map to map high-dimensional observations onto a low-dimensional grid while preserving neighborhood structure.

Suggested Dataset

UCI Wine: 178 samples, 13 chemical features, 3 cultivars (labels used only for visualization). Features are MinMax-scaled to $[0, 1]$.

Required Libraries

minisom, numpy, pandas, matplotlib.

Content, result plots and complete code listings for this experiment are filled in after the format review.

Brief Theory

Procedure

Reference Implementation

Evaluation / Expected Output

Viva / Discussion Questions

Complete Code Listing

Part VIII

Probabilistic Sequence Modeling: Hidden Markov Models

Experiment 16: Hidden Markov Model (HMM)

Dataset: DAX stock index 1991–1998 (`data/stock_index.csv`, `real EuStockMarkets`); 1,859 trading days
Key parameters: 3 Gaussian states, full covariance, 200 Baum–Welch (EM) iterations; Viterbi decoding
Headline result: EM converged; log-likelihood 6019.79; volatility-sorted Bull / Sideways / Bear regimes

Objective

Model a sequence using hidden states, transition probabilities and emission distributions; decode the most likely hidden-state path with the Viterbi algorithm.

Suggested Dataset

Real DAX stock index daily closes 1991–1998 (`data/stock_index.csv`, `R EuStockMarkets`), 1,859 trading days. The observation sequence is the daily return; the close price is used only for plotting.

Required Libraries

hmmlearn, numpy, pandas, matplotlib.

Content, result plots and complete code listings for this experiment are filled in after the format review.

Brief Theory

Procedure

Reference Implementation

Evaluation / Expected Output

Viva / Discussion Questions

Complete Code Listing

Part IX

Margin-Based Learning: Support Vector Machines

Experiment 17: Support Vector Machine (SVM)

Dataset: Breast Cancer (569×30 ; 2 features for the kernel view, all 30 for accuracy) and Old Faithful geyser (`data/geyser.csv`, 272 eruptions)

Key parameters: SVC: RBF kernel, $C = 1.0$, $\gamma = \text{'scale'}$; SVR: RBF, $C = 10$, $\varepsilon = 0.3$

Headline result: SVC accuracy 0.9790 (96/426 support vectors); SVR R^2 0.8968 (103/272 support vectors)

Objective

Train SVM classifiers and investigate the effects of kernel, C and γ ; demonstrate Support Vector Regression with an ε -insensitive tube.

Suggested Dataset

(a) Breast Cancer (569×30): two features (mean radius, mean texture) for the 2D kernel view and all 30 features for the accuracy benchmark. (b) Old Faithful geyser (`data/geyser.csv`, 272 eruptions) for SVR.

Required Libraries

scikit-learn, numpy, pandas, matplotlib.

Content, result plots and complete code listings for this experiment are filled in after the format review.

Brief Theory

Procedure

Reference Implementation

Evaluation / Expected Output

Viva / Discussion Questions

Complete Code Listing

Part X

Generative / Foundation Models: Large Language Models

Experiment 18: Large Language Model (LLM) Experiment

Dataset: SMS Spam Collection (`data/sms_spam.csv`, 5,572 messages; balanced subset 400 train / 200 test) + DistilBERT / DistilGPT2

Key parameters: Fine-tune: fresh 2-class head, 2 epochs, AdamW learning rate 5×10^{-5} , batch 16, max length 64

Headline result: Held-out accuracy 0.9600; spam F1 0.9592; greedy vs. nucleus generation demonstrated

Objective

Demonstrate a reproducible LLM workflow for text classification and generation while understanding tokenization, inference and evaluation.

Suggested Dataset

DistilBERT (SST-2) for tokenization and sentiment, DistilGPT2 for generation, and the real SMS Spam Collection (`data/sms_spam.csv`) for fine-tuning. A balanced subset (300 ham + 300 spam) is split into 400 training and 200 stratified test messages.

Required Libraries

transformers, torch, pandas.

Content, result plots and complete code listings for this experiment are filled in after the format review.

Brief Theory

Procedure

Reference Implementation

Evaluation / Expected Output

Viva / Discussion Questions

Complete Code Listing

Part XI

Non-Parametric Regression: Generalized Regression Neural Network

Experiment 19: Generalized Regression Neural Network (GRNN)

Dataset: Real motorcycle impact data (`data/motorcycle.csv`, `R MASS::mcycle`); 133 observations, time vs head acceleration

Key parameters: GRNN from scratch (NumPy); smoothing spread σ selected by 5-fold cross-validation over 0.02–1.0

Headline result: $\sigma = 0.08$ (CV RMSE 25.00 g); held-out R^2 0.7255, RMSE 22.78 g

Objective

Implement GRNN-style kernel regression from scratch and study the effect of the smoothing parameter σ with cross-validation.

Suggested Dataset

Real motorcycle impact data (`data/motorcycle.csv`, `R MASS::mcycle`): 133 observations of time after impact (ms) versus head acceleration (g). Time is standardized before computing kernel distances.

Required Libraries

numpy, pandas, matplotlib, scikit-learn (base classes, KFold, metrics).

Content, result plots and complete code listings for this experiment are filled in after the format review.

Brief Theory

Procedure

Reference Implementation

Evaluation / Expected Output

Viva / Discussion Questions

Complete Code Listing

Part XII

Integrated Mini Project and Appendices

Integrated Mini Project: Comparative Machine Learning Study

Scope: Compare algorithms from different families on identical data, splits and seeds

Protocol: Fixed random seeds; untouched test sets; two or more metrics per task

Headline result: Gradient boosting leads tabular regression; ensembles tie on small classification data; density clustering finds real seismic zones without labels

Objective

For the final laboratory assessment, compare at least three algorithms from different families on one real-world dataset, with a controlled, explainable experiment rather than a search for the largest score.

Suggested Dataset

The manual's own experiments provide the comparison material: Heart Disease (classification), California Housing (regression), Mall Customers (clustering) and the USGS earthquake catalogue (density-based clustering).

Required Libraries

scikit-learn, xgboost, catboost, numpy, pandas, matplotlib.

Content, result plots and complete code listings for this experiment are filled in after the format review.

Workflow

Comparative Results

Discussion and Conclusions

Viva / Discussion Questions

Appendix A: Quick Reference Formulas

Concept	Formula / Definition
Euclidean distance	$d(x, y) = \sqrt{\sum_j (x_j - y_j)^2}$
K-means objective	$J = \sum_i \ x_i - \mu_{c_i}\ ^2$
Regression MAE	$\text{MAE} = \frac{1}{n} \sum_i y_i - \hat{y}_i $
Regression RMSE	$\text{RMSE} = \sqrt{\frac{1}{n} \sum_i (y_i - \hat{y}_i)^2}$
R^2	$R^2 = 1 - \text{SS}_{\text{res}} / \text{SS}_{\text{tot}}$
Precision	$\text{TP} / (\text{TP} + \text{FP})$
Recall	$\text{TP} / (\text{TP} + \text{FN})$
F1	$2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$
Sigmoid	$\sigma(z) = 1 / (1 + e^{-z})$
SVM margin idea	Maximize margin while penalizing violations via C
GRNN kernel	$w_i(x) = \exp(-\ x - x_i\ ^2 / (2\sigma^2))$

Appendix B: Reproducibility Checklist

- Python version recorded.
- Package versions recorded.
- Random seed recorded where supported (all experiments use seed 42).
- Dataset source and version recorded (public datasets committed under `data/`).
- Preprocessing steps recorded (scaling fitted on training data only; haversine radians for geospatial data).
- Train/validation/test protocol recorded (stratified, chronological, early-stopping split, 5-fold CV).
- Hyperparameters recorded.
- Evaluation metrics defined before final testing.
- Plots and tables saved with meaningful names.
- Code runs from a clean environment or requirements file.

Appendix C: Suggested Assessment Rubric

Component	Suggested Weight
Pre-lab preparation / algorithm understanding	10%
Correct implementation	25%
Experimental design and preprocessing	15%
Evaluation and visualization	15%
Analysis and interpretation	20%
Report quality / reproducibility	10%
Viva / discussion	5%

Appendix D: Student Experiment Record

Field	Student Entry
Name	
ID	
Section	
Experiment No.	
Date	
Dataset	
Algorithm / Version	
Key Hyperparameters	
Random Seed	
Main Result	
Observation	
Instructor Signature	